

A REPORT OF 4-6 MONTHS INDUSTRIAL TRAINING

at

MARUTI SUZUKI INDIA LIMITED

SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENT
FOR
THE AWARD OF DEGREE OF

BACHELOR OF TECHNOLOGY
(Electronics and Computer Engineering)



JAN-JUNE 2026

SUBMITTED BY:

TARUN JYOTI

12205023

**DEPARTMENT OF ELECTRONICS AND COMMUNICATION
ENGINEERING**

PUNJABI UNIVERSITY, PATIALA

CERTIFICATE



TO WHOM IT MAY CONCERN

This is to certify that Tarun Jyoti a student of Electronics and Computer Engineering is currently pursuing training at Maruti Suzuki India Limited in the field of "Full Stack Development using Java Microservices and React".

The training commenced on 22 Jan 2026 and is expected to be completed on 22 Jun 2026.

During the training period so far, the student has been sincere, hardworking, and has demonstrated good conduct.

We wish them success in their future endeavors.

Date: 23/4/2026

Place: Gurugram



Authorized Signatory

Himani Singla

Associate General Manager

Maruti Suzuki India Limited

MARUTI SUZUKI INDIA LIMITED

Head Office :
Maruti Suzuki India Limited,
1, Nelson Mandela Road, Vasant Kunj,
New Delhi-110070, India
Tel: 011-46781000, Fax: 011-46150275/46150276
Email id : contact@maruti.co.in, www.marutisuzuki.com

Gurgaon Office :
Maruti Suzuki India Limited,
Old Palam Gurgaon Road,
Gurgaon - 122015, Haryana, India.
Tel: 0124-2346721-30, Fax: 0124-2341304

Manesar Plant :
Maruti Suzuki India Limited,
Plot No. 1, Phase- 3A, IMT Manesar,
Gurgaon - 122051, Haryana, India.
Tel: 0124-4884000, Fax: 0124-4884199

CIN : L34103DL1981PLC011375

CANDIDATE’S DECLARATION

I hereby certify that the work which is being presented in the B. Tech Industrial Project Report entitled “**Microservices-Based Full-Stack C2C Car Resale Platform**”, in partial fulfillment of the requirements for the award of the Bachelor of Technology in Electronics & Computer Engineering and submitted to the Department of Electronics & Communication Engineering of Punjabi University, Patiala is an authentic record of my work carried out during a period from **22 JAN 2026 to 22 JUN 2026**.

Tarun Jyoti

Signature of the Student

The summer training Viva-Voce Examination of _____
_____ has been held on
_____ and accepted.

Signature of the Examiner

Acknowledgment

“This journey reflects the transformation of academic learning into professional capability.”

The successful completion of this project marks a significant milestone in my academic journey, and it would not have been possible without the guidance, support, and encouragement of several individuals and organizations.

I express my sincere gratitude to Maruti Suzuki for providing me with the invaluable opportunity to undertake my industrial training. The exposure to real-world systems, industry practices, and professional workflows has greatly enhanced my technical knowledge and practical understanding. This experience has played a crucial role in bridging the gap between academic learning and industry expectations.

I am deeply thankful to my mentor, **Mr. Ramachandran Kumar**, for his constant guidance, insightful feedback, and unwavering support throughout the internship. His expertise, mentorship, and encouragement helped me navigate challenges effectively and contributed significantly to the successful completion of this project. His approach toward problem-solving and system design has left a lasting impact on my professional development.

I would also like to extend my appreciation to all the team members and staff at Maruti Suzuki who contributed directly or indirectly to my learning experience. Their collaborative spirit, willingness to share knowledge, and professional environment made this journey both enriching and inspiring.

Furthermore, I express my heartfelt thanks to the faculty and the Head of the Department of Electronics and Communication at Punjabi University for their continuous support and for facilitating this opportunity. Their guidance ensured a smooth and structured training experience.

Lastly, I am deeply grateful to my family for their constant encouragement, patience, and belief in me. Their support has been the foundation of my achievements. I also thank the Almighty for providing me with the strength and determination to successfully complete this project.

INDEX

PART-1 INDUSTRIAL TRAINING REPORT.....	1
CHAPTER -1.....	2
1.1 INTRODUCTION TO ORGANIZATION.....	2
1.2 TECHNOLOGIES LEARNT.....	6
1.2.1 Software Development Life Cycle (SDLC)	6
1.2.2 SDLC Models	8
1.2.3 Agile Execution and Sprint-Based Development	10
1.2.4 JIRA (Project Management Tool).....	10
1.2.5 Spring Boot Framework.....	12
1.2.6 System Architecture.....	13
1.2.7 Business Logic and Rules Engine.....	15
1.2.8 Devops	16
1.2.9 API Testing	20
1.2.10 Artifact Versioning	21
1.2.11 SBOM	22
PART -2 PROJECT	35
CHAPTER -2 PROJECT DETAILS.....	36
2.1 INTRODUCTION TO THE PROJECT	36
2.2 BLOCK DIAGRAM OF THE PROJECT	47
2.2.1 Block Diagram of Microservices-Based C2C Car Resale Platform	47
2.2.2 Microservices Interaction Diagram.....	48
2.2.3 Database ER Diagram.....	49
2.3 FLOWCHART OF THE PROJECT.....	50
2.4 CODING OF THE PROJECT.....	55
2.5 SNAPSHOT OF THE PROJECT.....	68
2.6 APPLICATION OF THE PROJECT	72
2.7 REFERENCES.....	73

PART-1 INDUSTRIAL TRAINING REPORT



MARUTI SUZUKI INDIA LIMITED (MSIL)

Ground Floor, Platinum Tower, Udyog Vihar Phase-1, Sector 20 ,Gurugram
Haryana – 122016, India

CHAPTER -1

1.1 INTRODUCTION TO ORGANIZATION

1.1.1 COMPANY PROFILE –MARUTI SUZUKI INDIA LIMITED

Maruti Suzuki India Limited is the Indian subsidiary of Japanese automaker Suzuki Motor Corporation. It is the largest automobile manufacturer in India, specialising in small cars. The company was established by the Government of India as Maruti Udyog Limited in February 1981 as a joint venture with Suzuki, the latter becoming the first Japanese automaker, as well as the first major foreign automaker, to invest in India. Maruti opened its first production facility in Gurugram, Haryana, in 1982. Initially, Maruti was majority-owned by the Indian government, with Suzuki only taking a 26% stake during its establishment in 1982.



Fig. 1.1 Initial Phase of Maruti Suzuki Automobile Production in India

The name Maruti Suzuki India Limited originates from an earlier initiative led by Sanjay Gandhi, the son of former Prime Minister Indira Gandhi. During the early 1970s, the Government of India aimed to develop an affordable “people’s car” to make automobile ownership accessible to the middle class. Sanjay Gandhi, who had received automobile training at Rolls-Royce in the United Kingdom, initiated a project to manufacture small cars in India. In 1971, he established Maruti Motors Limited and acquired land in Gurugram, Haryana to set up a manufacturing facility. The project aimed to produce an affordable indigenous car for the Indian market. Although the original plan was to sell the car at around ₹8,000, the estimated cost later increased. However, the proposed vehicle was still expected to be cheaper than other cars available in India at that time, supporting the vision of creating

a mass-market automobile. The first prototype of the Maruti car was completed in 1972, with production anticipated to commence by April 1973, as promised by Sanjay Gandhi.

1.1.2 Headquarters

The corporate headquarters of Maruti Suzuki India Limited is located in New Delhi, India. This headquarters manages the company's strategic planning, corporate administration, marketing operations, and nationwide business activities. The company operates major production and operational facilities in **Gurugram**, Haryana and **Manesar**, Haryana, which serve as important centers for manufacturing and operational management. Maruti Suzuki is a subsidiary of Suzuki Motor Corporation, whose global headquarters is located in **Hamamatsu**, Japan. The parent company supervises global business operations, technological development, and international expansion.



Fig. 1.2 Manufacturing Facility of Maruti Suzuki India Limited

1.1.3 Tie-up with Suzuki

In 1982, the Government of India entered into a joint venture with Suzuki Motor Corporation of Japan to establish **Maruti Udyog Limited**, with the objective of producing affordable and reliable cars for the Indian market. Initially, Suzuki acquired a 26% equity stake in the company, while the Government of India held the majority share. Over time, Suzuki gradually increased its ownership and became the majority stakeholder. This partnership brought advanced automobile technology, modern production techniques, and management expertise to India. As a result of this collaboration, the company launched its first car, the

Maruti 800, in 1983, which quickly became one of the most popular cars in India due to its affordability, fuel efficiency, and reliability.

The strategic alliance between Maruti and Suzuki significantly contributed to the modernization of the Indian automobile industry and played a key role in making passenger cars accessible to a large segment of the population.

1.1.4 Products and Services

Maruti Suzuki India Limited offers a diverse portfolio of vehicles designed to meet the needs of various customer segments. The company is widely recognized for its focus on fuel-efficient and cost-effective automobiles.

The major product categories include:

- Hatchbacks (e.g., Alto, Swift, WagonR)
- Sedans (e.g., Dzire, Ciaz)
- Sport Utility Vehicles (SUVs) (e.g., Brezza, Grand Vitara)
- Hybrid and eco-friendly vehicles

In addition to manufacturing automobiles, the company provides a comprehensive range of services, including:

- After-sales service and maintenance
- Supply of genuine spare parts
- Customer support services
- A widespread service network across the country

These offerings contribute to maintaining strong customer satisfaction and brand loyalty.

In conclusion, Maruti Suzuki India Limited is a leading organization that has significantly contributed to the growth and development of the Indian automobile industry. The company's emphasis on innovation, quality assurance, and customer satisfaction has helped it achieve a strong market presence. Its adoption of modern technologies and efficient management practices reflects its commitment to excellence. From a learning perspective, the organization provides an excellent platform for gaining practical knowledge and understanding real-world applications of engineering concepts. The exposure gained through this organization has been highly beneficial in bridging the gap between theoretical learning and industry practices.

1.1.5 COMPANY OUTCOME

Maruti Suzuki India Limited has established itself as a dominant force in the Indian automotive industry through its focus on innovation, large-scale manufacturing, and customer-centric product development. Over the years, the company has played a significant role in transforming the Indian automobile sector by introducing modern manufacturing technologies, efficient supply chain systems, and high-quality yet affordable vehicles.

From an industrial perspective, Maruti Suzuki focuses on continuous improvement in production efficiency, quality management, and technological advancement. The company integrates advanced manufacturing techniques such as automation, robotics, and data-driven quality control systems to maintain high standards in vehicle production.

Vision

The vision of Maruti Suzuki India Limited is to become a globally recognized leader in the automobile industry by delivering innovative, safe, and environmentally sustainable mobility solutions. The company aims to transform the mobility experience of customers by integrating advanced technology, efficient manufacturing practices, and superior product quality.

From an industrial perspective, the company's vision focuses on building world-class manufacturing facilities, developing next-generation automotive technologies, and maintaining the highest standards of operational efficiency. Maruti Suzuki strives to adopt smart manufacturing techniques, digital technologies, and Industry 4.0 practices to enhance productivity and ensure consistent product quality.

Mission

The mission of Maruti Suzuki India Limited is to provide high-quality, affordable, and technologically advanced vehicles that meet the evolving needs of customers. The company focuses on maintaining excellence in manufacturing, innovation, and customer service.

Maruti Suzuki aims to achieve this mission by continuously improving its production processes, investing in research and development, and adopting modern engineering practices. The company also focuses on building strong relationships with customers, suppliers, and stakeholders to ensure long-term growth and success.

1.2 TECHNOLOGIES LEARNT

1.2.1 Software Development Life Cycle (SDLC)

The Software Development Life Cycle (SDLC) is a structured framework used by software organizations to design, develop, and test high-quality software. It defines the entire process of software production, from the initial idea to the final deployment and maintenance.

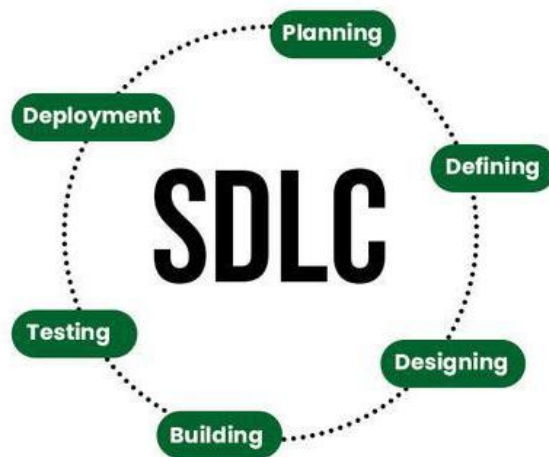


Fig. 1.3 Software Development Life Cycle (SDLC)

Stages of the Software Development Life Cycle

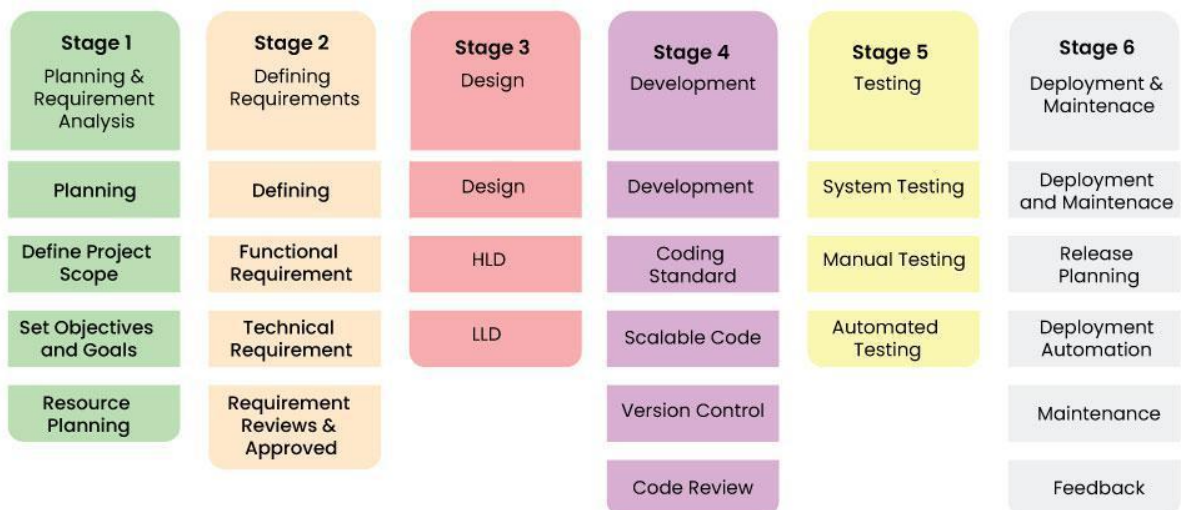


Fig. 1.4 Stages of Software Development Life Cycle

1. Planning and Requirement Analysis

This phase involves understanding the project objectives, defining scope, and analyzing requirements.

Project Implementation:

In the project, the requirements of the car marketplace system were analyzed, including features such as user management, car listing, and booking functionality. Project goals and scope were clearly defined.

2. Defining Requirements

In this phase, functional and technical requirements are documented and validated.

Project Implementation:

Functional requirements such as user registration, login, car posting, and booking were identified. Technical requirements included the use of Spring Boot for backend development and MySQL for database management.

3. Design

The design phase focuses on system architecture and component structure.

Project Implementation:

The system was designed using a microservices architecture.

4. Development

In this phase, the actual coding of the application is performed.

Project Implementation:

The application was developed using Spring Boot. Different microservices were implemented, and version control was managed using Git and GitHub to track changes.

5. Testing

Testing ensures that the application functions correctly and is free from defects.

Project Implementation:

Testing was carried out to validate the functionality of APIs and services. Code quality and security were ensured using SonarQube, and issues were resolved during development.

6. Deployment and Maintenance

This phase involves deploying the application and maintaining it after release.

Project Implementation:

The application was containerized using Docker and deployed in a local environment. Jenkins was used to automate the build and deployment process. Continuous updates and improvements were made based on testing and feedback.

1.2.2 SDLC Models

Different SDLC models are used depending on the project requirements and development approach. Some of the commonly used SDLC models are:

1. Waterfall Model

The Waterfall model is a linear and sequential approach to software development. In this model, each phase must be completed before moving to the next phase. It is simple and easy to understand. Suitable for small projects with fixed requirements. There is a limitation difficult to go back to previous stage.

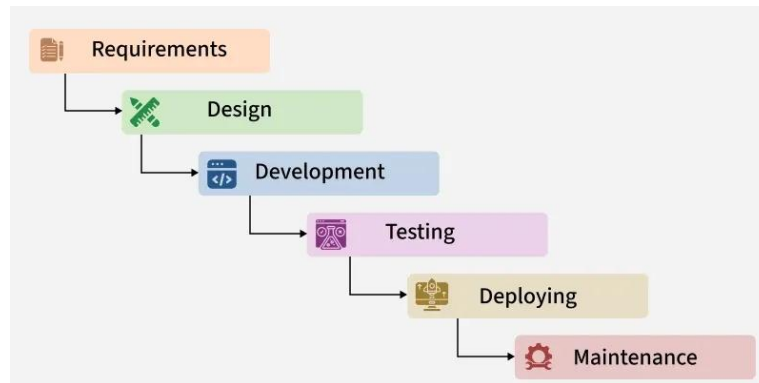


Fig. 1.5 Phases of Software Development Life Cycle

2. Agile Model

The Agile model is an iterative and flexible approach where development is carried out in small increments. It focuses on continuous improvement, collaboration, and adaptability. This is Iterative development, faster delivery of features and Continuous feedback and improvement. It enables teams to respond quickly to changes and ensures that the final product meets user expectations effectively. Agile methodology focuses on delivering value through continuous interaction between development teams and clients. It ensures better transparency, faster problem resolution, and improved product quality. It enables efficient handling of changing requirements and improves overall project management.

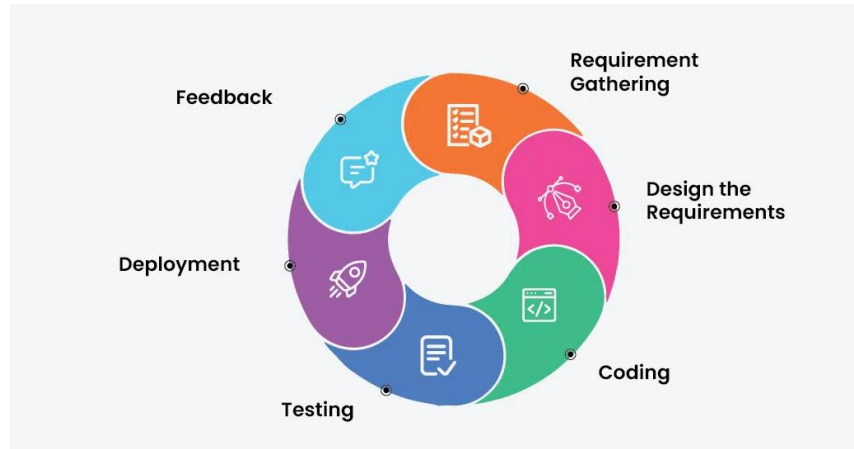


Fig. 1.6 Iterative SDLC Process Model

Agile Implementation in the Project

In this project, the Agile methodology was adopted to ensure efficient and flexible development. The entire system was developed in a step-by-step manner by dividing the work into smaller units called sprints. Each sprint focused on completing a specific set of features of the application. The development process in each sprint included requirement gathering, design, coding, testing, and deployment of the module.

The project followed an iterative approach where feedback was continuously incorporated to improve the system. This helped in identifying issues early and making necessary improvements without affecting the overall development process. Daily progress was tracked, and tasks were updated regularly to ensure smooth workflow and timely completion of the project.

Sprint Structure in the Project

Each sprint in the project consisted of the following stages:

1. **Planning Phase**
In this phase, tasks were identified and prioritized based on project requirements.
2. **Development Phase**
The features were implemented using technologies such as Java and Spring Boot.
3. **Testing Phase**
APIs and functionalities were tested using tools like Postman to ensure correctness.
4. **Review Phase**
The completed work was reviewed, and feedback was collected for improvements.

1.2.3 Agile Execution and Sprint-Based Development

During the course of my industrial training, I gained in-depth exposure to Agile software development methodology, which is widely adopted in modern software organizations for iterative and incremental delivery of applications. The project development process was structured into multiple sprints, each representing a short development cycle focused on delivering a specific set of functionalities.

The project tasks were managed using Jira, where requirements were broken down into **epics and user stories**. Each user story represented a specific functionality such as user registration, car listing, bidding, or booking. These stories were prioritized in the product backlog based on business value and implementation complexity.

A structured approach was followed for sprint execution:

- **Sprint Planning:** Tasks were selected from the backlog and assigned based on priority and feasibility.
- **Definition of Ready (DoR):** A task was considered ready only when it had clearly defined requirements, acceptance criteria, and necessary dependencies resolved.
- **Development and Execution:** Each sprint involved designing, coding, testing, and integrating the assigned features.
- **Definition of Done (DoD):** A task was marked complete only after successful implementation, testing, and validation against acceptance criteria.
- **Progress Tracking:** Jira boards were used to track task status across stages such as *To Do*, *In Progress*, and *Done*.
- **Sprint Review and Feedback:** Weekly presentations were conducted to demonstrate completed features and gather feedback for continuous improvement.

This iterative approach ensured flexibility, early issue detection, and continuous enhancement of the system.

1.2.4 JIRA (Project Management Tool)

Jira is a widely used project management and issue tracking tool developed by Atlassian. It is commonly used in Agile environments to manage tasks, track progress, and ensure efficient project execution.

The name "Jira" is derived from "Gojira," which is the Japanese name for Godzilla. It originated as an internal nickname used by developers at Atlassian while building a bug-tracking tool, as a reference to its predecessor, Bugzilla. Over time, this informal nickname became the official

product name, and Jira has since grown into one of the most widely adopted project management tools in the software industry

Use of JIRA in the Project

In this project, Jira was used to effectively implement Agile methodology and manage development activities.

- Tasks were created in the form of user stories and issues
- Work was organized into sprints for systematic development
- Progress of tasks was tracked using Agile boards
- Tasks were categorized into stages such as:
 - To Do
 - In Progress
 - Done

Jira helped in maintaining transparency and ensured that all team members were aware of their responsibilities and deadlines.

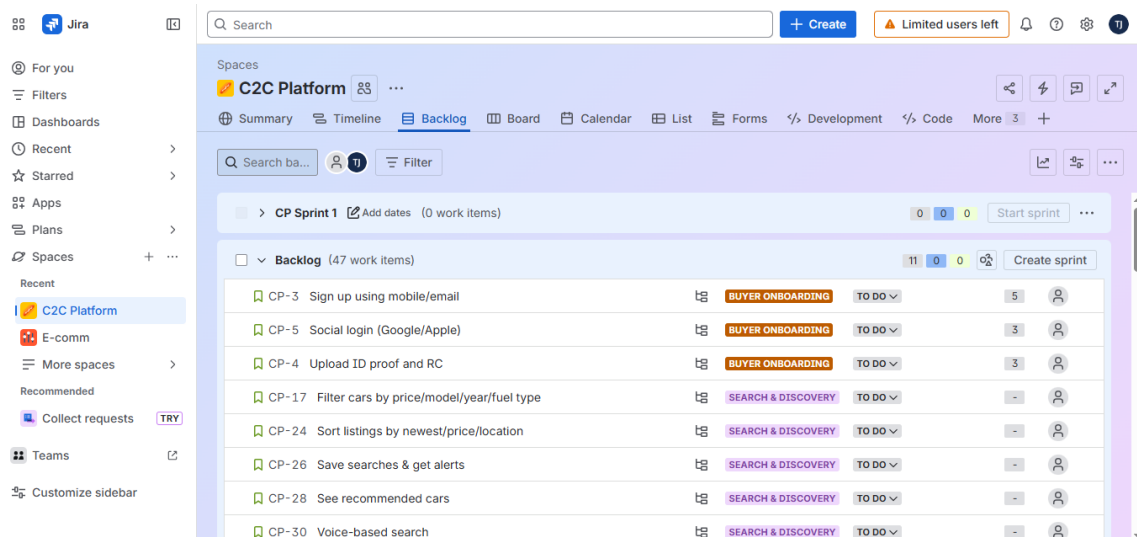


Fig. 1.7 Jira Backlog for C2C Platform

The above figure represents the Jira dashboard used for managing the project tasks and development workflow. In the screenshot, the project titled “**C2C Platform**” is displayed, which contains all the tasks related to the development of the application.

The backlog section shows a list of all user stories and tasks that are yet to be completed. These tasks include functionalities such as user registration, social login, uploading documents, filtering cars, and sorting listings.

Each task is assigned a unique identifier (e.g., CP-3, CP-5) and is categorized under different modules such as Buyer Onboarding and Search & Discovery, which helps in better organization of work. The status of each task is also visible, indicating whether the task is in “To Do”, “In Progress”, or completed stage. This helps in tracking the real-time progress of the project.

Sprint Timeline and Project Planning

In this project, Jira was used not only for task management but also for planning and tracking the project timeline. The timeline view provides a visual representation of how different project activities and epics are scheduled over a period of time. It helps in understanding the sequence of development, managing deadlines, and ensuring proper distribution of work across sprints.

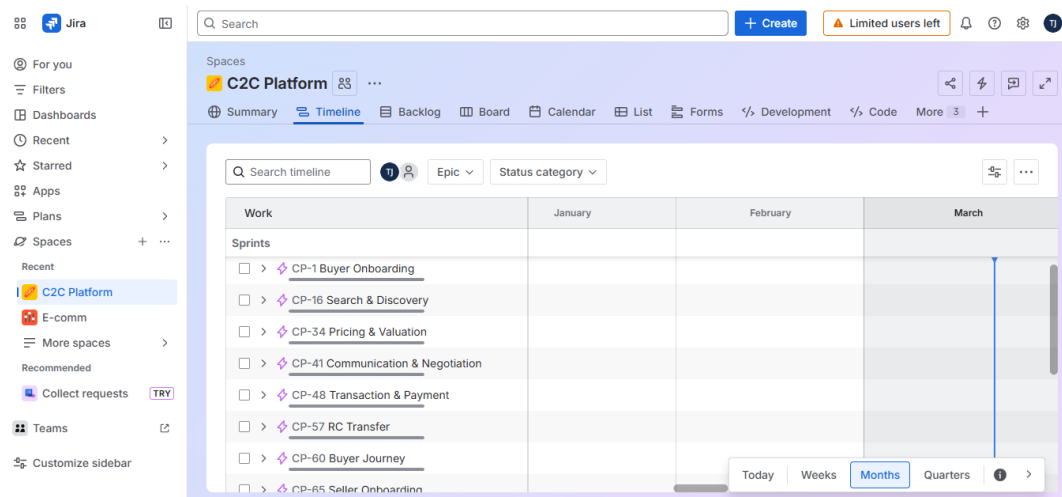


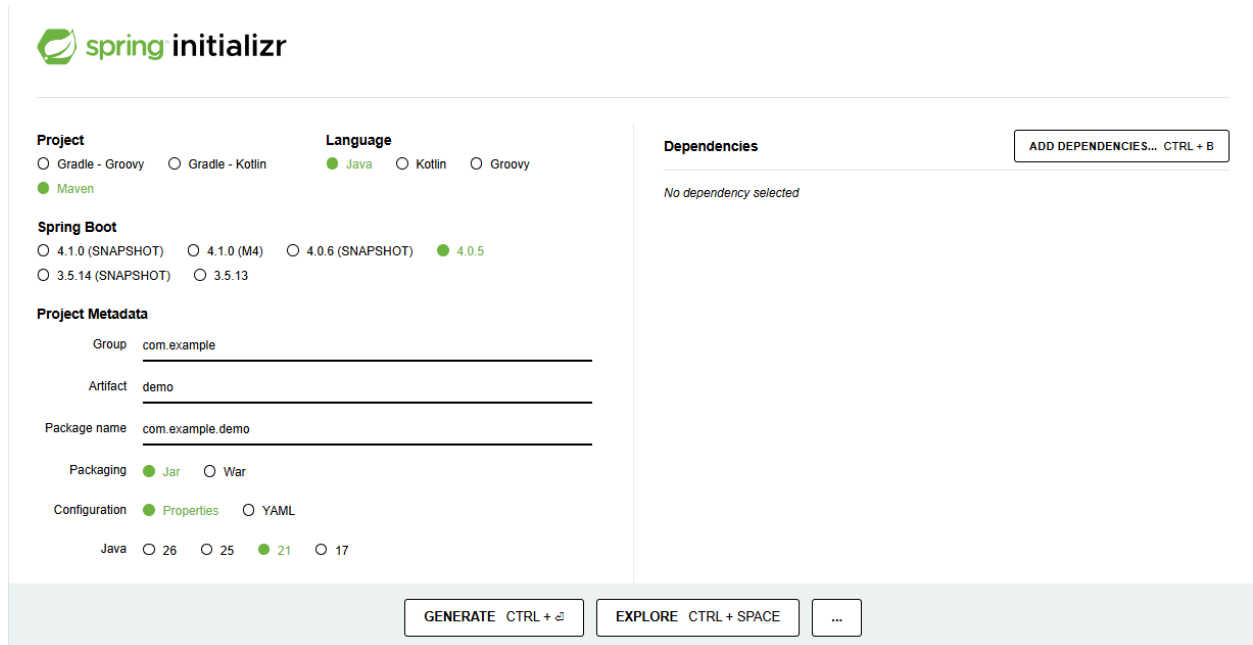
Fig. 1.8 Jira Timeline and Sprint Planning

The above figure shows the timeline view of the project in Jira. It displays various epics such as Buyer Onboarding, Search & Discovery, Pricing & Valuation, Communication & Negotiation, and Transaction & Payment. Each epic represents a major functional module of the system and is scheduled across a specific time period. The timeline is divided into months, allowing better visualization of when each module is planned and developed. This helps in tracking progress, managing deadlines, and ensuring that the project stays on schedule.

1.2.5 Spring Boot Framework

Spring Boot is an open-source Java-based framework used to develop stand-alone, production-ready applications with minimal configuration. It is built on top of the Spring

Framework and simplifies the development of enterprise-level applications by reducing boilerplate code and configuration complexity. Spring Boot provides a streamlined way to create RESTful web services and microservices, making it highly suitable for modern application development.



The screenshot shows the Spring Initializr web interface. At the top is the 'spring initializr' logo. Below it, the configuration is organized into sections: 'Project' with radio buttons for 'Gradle - Groovy', 'Gradle - Kotlin', 'Maven' (selected), and 'Language' with radio buttons for 'Java' (selected), 'Kotlin', and 'Groovy'. The 'Spring Boot' section has radio buttons for versions: '4.1.0 (SNAPSHOT)', '4.1.0 (M4)', '4.0.6 (SNAPSHOT)', '4.0.5' (selected), and '3.5.14 (SNAPSHOT)', '3.5.13'. The 'Project Metadata' section contains text input fields for 'Group' (com.example), 'Artifact' (demo), and 'Package name' (com.example.demo). Below these are 'Packaging' radio buttons for 'Jar' (selected) and 'War', and 'Configuration' radio buttons for 'Properties' (selected) and 'YAML'. At the bottom of the metadata section are 'Java' version radio buttons: '26', '25', '21' (selected), and '17'. On the right, the 'Dependencies' section has a button 'ADD DEPENDENCIES... CTRL + B' and the text 'No dependency selected'. At the bottom of the form are three buttons: 'GENERATE CTRL + G', 'EXPLORE CTRL + SPACE', and an ellipsis button '...'.

Fig. 1.9 Spring Initializr Configuration

Usage of Spring Boot in the Project

In this project, Spring Boot was used to develop the backend of the C2C car marketplace application. It was primarily used for creating REST APIs and implementing business logic. The project was structured using microservices architecture, where each service was developed as a separate Spring Boot application. Spring Boot helped in handling HTTP requests, managing dependencies, and connecting with the database efficiently. Additionally, its embedded server feature allowed the application to run independently without requiring external deployment, making development and testing easier.

1.2.6 System Architecture

Monolithic architecture is a traditional software design approach in which the entire application is developed as a single, unified system. In this architecture, all components are interdependent, and communication between them happens internally within the same application. In general, monolithic systems are simple to develop and suitable for small-scale applications. Since all modules are part of a single application, testing and deployment are straightforward during the initial stages. However, as the application grows, the codebase

becomes large and difficult to manage. Any modification in one part of the system requires rebuilding and redeploying the entire application.

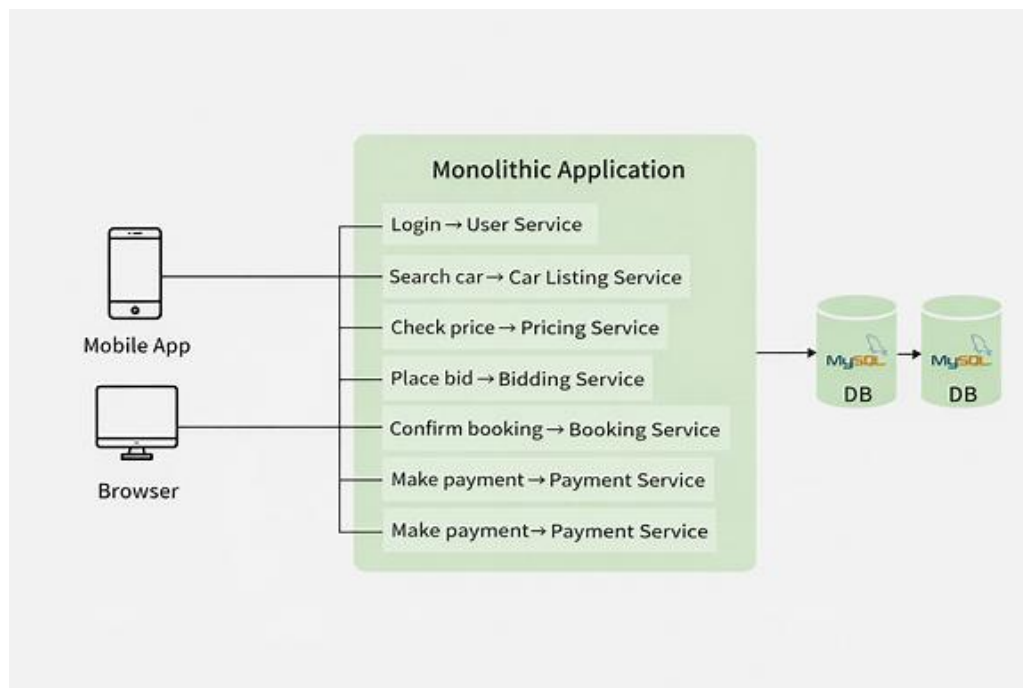


Fig. 1.10 Monolithic Architecture

In the context of the MarutiXchange platform, a monolithic approach would mean that all functionalities such as user registration, car listing, pricing, bidding, booking, and payment are implemented within a single application. The system would use a centralized database to store all types of data, including user information, car details, booking records, and payment transactions.

Although this approach simplifies initial development, it introduces several limitations. The system becomes difficult to scale, as the entire application must be scaled instead of individual components. Maintenance becomes complex due to tight coupling between modules. Additionally, failure in one part of the system can affect the entire application. Due to these limitations, monolithic architecture is not suitable for large and complex applications like MarutiXchange.

Microservices Architecture

Microservices architecture is a modern software design approach in which an application is divided into multiple small, independent, and loosely coupled services. Each service is responsible for a specific functionality and communicates with other services through

RESTful APIs. This architecture promotes modular development and allows each service to be developed, deployed, and scaled independently.

In general, microservices architecture consists of multiple layers, including the client layer, communication layer, service layer, and data layer. The client layer includes user interfaces such as mobile applications and web browsers. The communication layer, typically implemented using REST APIs, handles incoming requests and routes them to appropriate services. Each microservice operates independently and manages its own data, ensuring better scalability and flexibility.

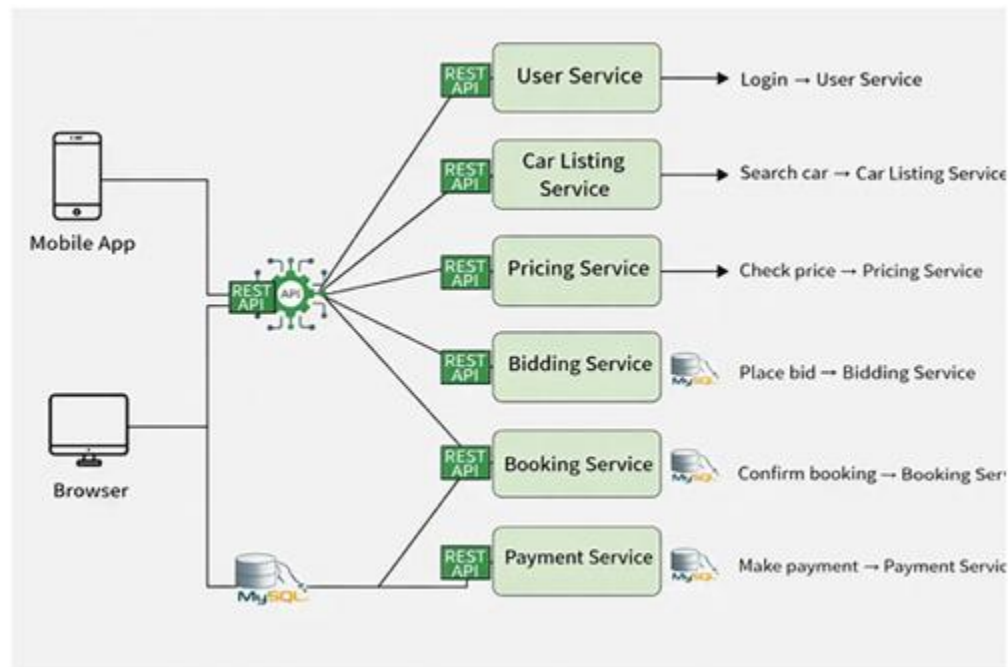


Fig.1.11 Microservices Architecture for a Car Marketplace Application

1.2.7 Business Logic and Rules Engine

A rules engine is a software component used to define, manage, and execute business rules in an application. It allows the system to make decisions based on predefined conditions without hardcoding the logic into the application code. In general, a rules engine evaluates input data against a set of rules and produces an output based on those conditions.

The primary advantage of using a rules engine is the separation of business logic from the application logic. This makes the system more flexible, maintainable, and scalable, as rules can be modified without changing the core codebase. Rules are typically defined using simple **“if-then” conditions**, which make them easier to understand and manage.

In modern applications, rules engines are widely used in scenarios such as pricing systems, validation mechanisms, recommendation systems, and workflow management, where dynamic decision-making is required.

Drools: Business Rules Management System

Drools are an open-source Business Rules Management System (BRMS) used to implement rule-based logic in software applications. It provides a powerful and flexible platform for defining and executing business rules separately from the application code. Drools uses a rule-based approach where rules are written in a declarative format using Drools Rule Language (DRL). These rules are evaluated by the Drools inference engine, which processes the input data (facts) and determines which rules should be executed.

Rules Engine Implementation in MarutiXchange

In the MarutiXchange system, rules engine logic was implemented to manage business operations efficiently and dynamically. Instead of hardcoding conditions, rule-based logic was used to handle key functionalities such as pricing, bidding, and booking. For instance, in the pricing module, rules are applied to determine the value of a car based on parameters such as model, manufacturing year, and condition. In the bidding module, rules ensure that each new bid is higher than the current bid and that bidding occurs within a valid time frame. Similarly, in the booking process, rules are used to validate user eligibility and booking conditions.

The following examples illustrate how rules are applied in the system:

- **Pricing Rule:**
If the car age is greater than a certain limit, the price is reduced accordingly.
- **Bidding Rule:**
If a new bid is higher than the current highest bid, it is accepted; otherwise, it is rejected.
- **Booking Rule:**
A booking is allowed only if the car is available and the user is authenticated.

1.2.8 Devops

DevOps is a combination of two words: "Development" and "Operations." It's a modern approach where software developers and software operations teams work together throughout the entire software life cycle. Before DevOps, software delivery was slow and manual. Separate teams handled coding, server setup, testing, and deployment, leading to delays and frequent errors due to lack of automation.

With DevOps, the process is fast, automated, and collaborative. Using tools like Git, Jenkins, Docker, and Kubernetes, teams can build, test, and deploy code continuously, enabling deployment in hours instead of days.

A key learning outcome of the training was the implementation of DevOps practices, particularly Continuous Integration and Continuous Deployment (CI/CD), which are essential for modern software development. A CI/CD pipeline was configured using Jenkins to automate the entire development lifecycle. The pipeline included the following stages:

- **Code Integration:** Source code changes were pushed to the repository and automatically triggered the pipeline.
- **Build Stage:** The application was built using Maven, ensuring proper dependency resolution and compilation.
- **Testing Stage:** Automated unit tests were executed to validate functionality and detect defects early.
- **Code Quality Analysis:** SonarQube was integrated to analyze code quality, detect vulnerabilities, and enforce coding standards.
- **Artifact Versioning:** Versioned build artifacts (JAR files) were generated to track application versions.
- **Deployment:** Docker was used to containerize applications, enabling consistent and reliable deployment across environments.

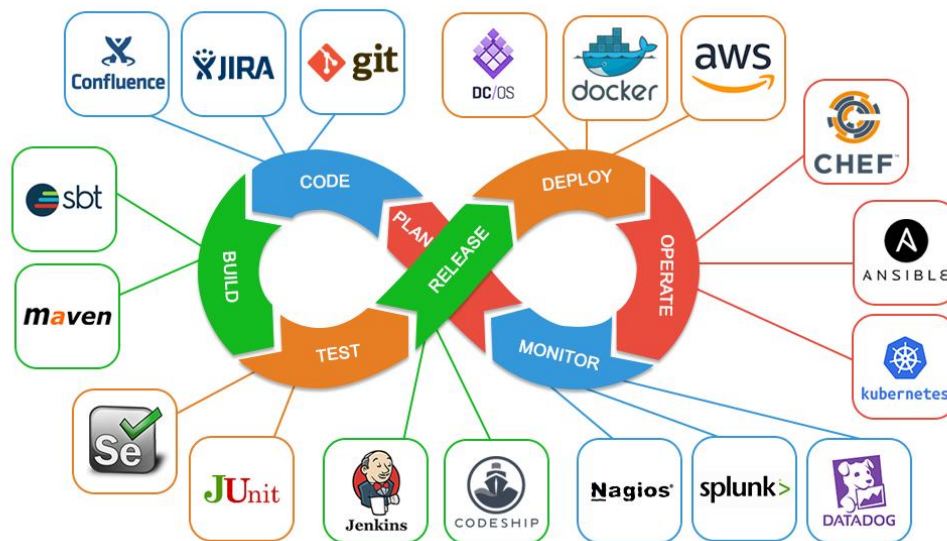


Fig. 1.12 Phases of the DevOps lifecycle and supporting technologies

DevOps Lifecycle

The DevOps lifecycle consists of several stages:

- **Planning** – Defining requirements and tasks
- **Development** – Writing and managing code
- **Build** – Compiling and packaging the application
- **Testing** – Verifying functionality and quality
- **Integration** – Merging code changes
- **Deployment** – Releasing the application
- **Monitoring** – Tracking performance and issues

Tools Used in DevOps

Jenkins

Jenkins is an open-source automation tool used for Continuous Integration (CI). It helps in automatically building and testing the application whenever changes are made to the codebase.

- Automates build process
- Integrates code changes
- Supports CI/CD pipelines

SonarQube

SonarQube is a static code analysis tool used to evaluate code quality and detect bugs, vulnerabilities, and code smells.

- Performs code analysis
- Ensures coding standards
- Provides quality gates

Testing

Unit Testing (JUnit)

Unit testing is a software testing technique in which individual components or units of a program are tested independently to ensure that they function correctly. A unit can be a function, method, or class that performs a specific task within the system. The primary objective of unit testing is to verify the correctness of each unit in isolation before integrating

it with other components. It helps in identifying errors at an early stage of development, thereby improving code quality and reducing debugging efforts..

Types of Unit Testing

Unit testing can be broadly classified into two types:

1. Manual Unit Testing

In manual unit testing, test cases are executed manually without the use of automation tools. The tester provides input values, observes the output, and verifies whether the results are correct.

- Suitable for small applications
- Time-consuming and less efficient
- Prone to human errors

2. Automated Unit Testing

In automated unit testing, test cases are written using testing frameworks and executed automatically. Tools like JUnit are used to validate the functionality of individual components.

- Faster and more efficient
- Reduces human effort
- Supports continuous integration
- Widely used in modern applications

In the MarutiXchange project, automated unit testing was used to ensure the correctness of different modules.

Techniques of Unit Testing

Black Box Testing

Black box testing is a method where the tester checks the functionality of the system without knowing the internal implementation. The tester provides input and verifies the output based on expected results.

In the MarutiXchange project, black box testing was used to test functionalities such as user login, car search, and booking.

White Box Testing

White box testing is a method where the tester has complete knowledge of the internal code structure. It focuses on testing the internal logic, conditions, and flow of the program.

In this project, white box testing was applied to verify internal logic such as bidding rules and pricing calculations.

Grey Box Testing

Grey box testing is a combination of both black box and white box testing. In this approach, the tester has partial knowledge of the system.

In the MarutiXchange project, grey box testing was used for modules like pricing and booking, where some understanding of internal logic was required.

1.2.9 API Testing

API testing is a type of testing used to verify the functionality, reliability, and performance of application programming interfaces (APIs). It ensures that communication between different components of the system works correctly. API testing is mainly considered a form of integration testing, as it validates the interaction between different services. It can also be viewed as black box testing since the internal implementation is not visible and testing is performed based on request and response.

API Testing in the Project

In the MarutiXchange project, API testing was performed using Postman to validate various endpoints of the system.

The following operations were tested:

- **GET requests** – To retrieve data such as car listings
- **POST requests** – To create new records such as user registration
- **PUT requests** – To update existing data
- **DELETE requests** – To remove data

The responses were verified based on:

- Status codes (200, 201, 400, etc.)
- Response data correctness
- API performance

1.2.10 Artifact Versioning

Artifact Versioning

Artifact versioning is the process of assigning unique version numbers to build outputs (artifacts) such as JAR or WAR files. It helps in identifying different versions of the application during development and deployment.

In software development, each time the code is updated and built, a new version of the artifact is created. This ensures proper tracking of changes and allows developers to manage different versions of the application effectively. It also helps in debugging, rollback, and maintaining consistency across different environments.

Version Format

A common versioning format used in software development is:

MAJOR.MINOR.PATCH

- **MAJOR version** is increased when there are major changes or breaking updates that are not backward compatible.
- **MINOR version** is increased when new features are added without breaking existing functionality.
- **PATCH version** is increased when minor bug fixes or improvements are made.

For example: **1.2.3**

- **1** → Major version
- **2** → Minor version
- **3** → Patch version

If a major update is made, the version may change from **1.2.3** → **2.0.0**.

If a new feature is added, it may change to **1.3.0**.

If only a bug is fixed, it may change to **1.2.4**.

Usage in Project

In the MarutiXchange project, artifact versioning was used during the build process with Maven. Each build generated a versioned artifact (JAR file), which helped in tracking application updates and managing deployments.

1.2.11 SBOM

Software Bill of Materials (SBOM)

Software Bill of Materials (SBOM) is a structured list that contains all the components, libraries, and dependencies used in a software application. It helps in identifying what software components are included in the system. In modern software development; applications depend on multiple third-party libraries. SBOM provides transparency by listing all these dependencies, including their versions and sources.

Importance of SBOM

- Helps in identifying security vulnerabilities
- Improves transparency of software components
- Helps in maintaining software compliance

Usage in DevOps

SBOM is commonly used in DevOps practices to track dependencies and ensure that the application is secure and free from vulnerable components.

Relevance to Project

In the MarutiXchange project, SBOM concept can be associated with dependency management through Maven, where all libraries and their versions are maintained. This helps in understanding and managing the components used in the application.

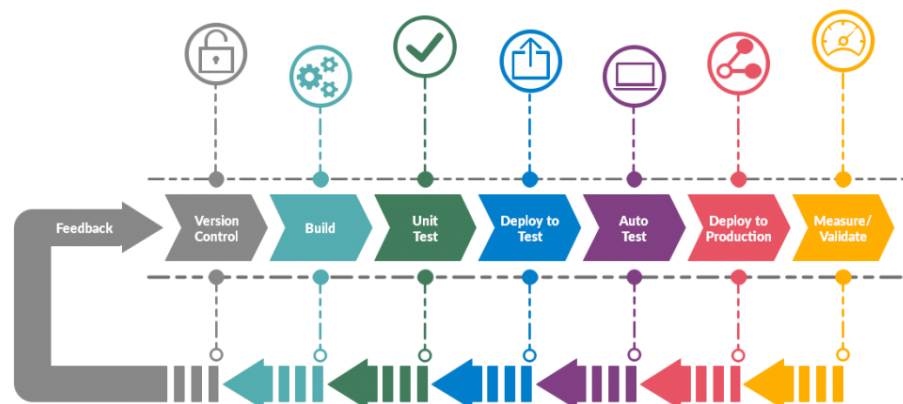


Fig. 1.13 Standardized DevOps Pipeline Architecture

DevOps Implementation in the Project

In the MarutiXchange project, DevOps practices were implemented to automate the development and integration process.

- The source code was managed using version control, and changes were integrated using Jenkins
- Jenkins was configured to automatically build the project and trigger analysis
- SonarQube was integrated with Jenkins to analyze code quality and ensure that the project meets predefined quality standards
- The quality gate mechanism ensured that only high-quality code was accepted

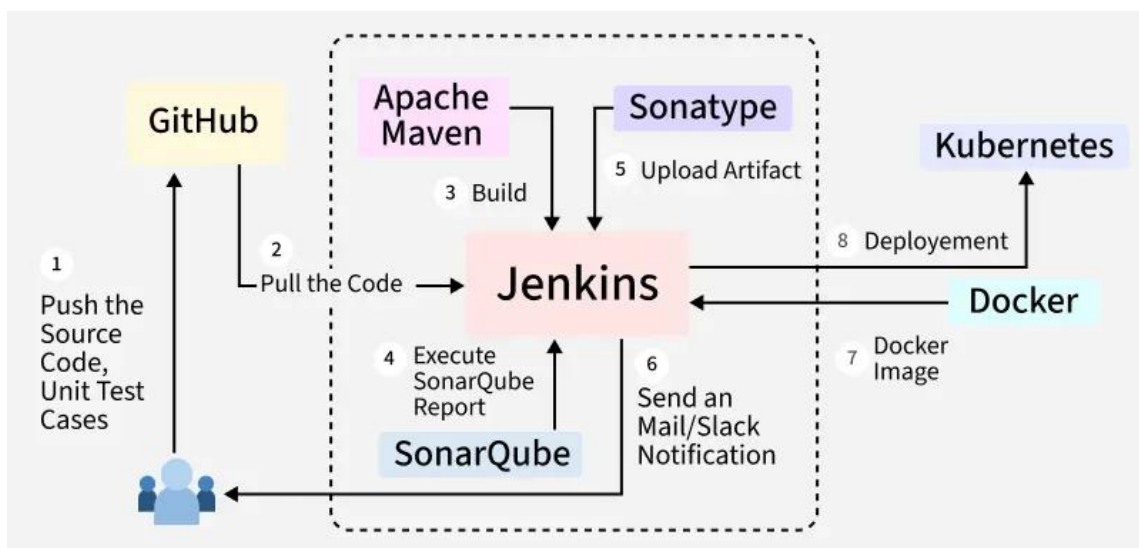


Fig. 1.14 Automated Jenkins CI/CD toolchain workflow

TOOLS TO BE LEARNT FOR DEVELOPMENT

1. Jira

Introduction

Jira is a widely used software development tool designed for project management and issue tracking. It is extensively adopted by software development teams to plan, monitor, and manage the entire lifecycle of software projects. Jira provides a centralized platform for managing tasks, bugs, and various types of work items, enabling teams to organize, prioritize, and track their progress efficiently. It was primarily used to create and manage tasks in the form of epics, user stories, and issues, which helped in breaking down the project

into smaller, manageable units. Jira enabled efficient sprint planning and task allocation, allowing the team to assign responsibilities and track progress throughout each development phase.



Fig. 1.15 Jira Software

Additionally, Jira was used for bug tracking and issue management, where defects and errors identified during development and testing were recorded, prioritized, and resolved systematically. The tool also provided workflow management features, ensuring that tasks moved through different stages such as To Do, In Progress, and Done in a structured manner.

Jira's dashboards and reporting features helped in monitoring project progress, analyzing team performance, and ensuring timely completion of tasks. Overall, Jira played a crucial role in improving collaboration, transparency, and productivity within the development process.

Usage of Jira in the Project

Jira was used to manage the project workflow by creating and tracking tasks, user stories, and bugs. It helped in organizing development activities through sprint planning and monitoring project progress.

2. VS Code

Visual Studio Code (VS Code) is a lightweight yet powerful source code editor developed by Microsoft. It supports multiple programming languages such as Java, JavaScript, Python, and TypeScript, making it highly versatile for modern application development. VS Code is widely used in the software industry due to its speed, flexibility, and extensive ecosystem of extensions that enhance developer productivity. It provides an efficient environment for writing, debugging, and managing code with minimal system overhead.

The editor provides powerful debugging capabilities, enabling developers to set breakpoints, inspect variables, and execute code step-by-step. This helps in identifying and resolving

issues efficiently during development. Additionally, VS Code includes an integrated terminal that allows developers to execute commands, run applications, and manage project builds without leaving the editor.

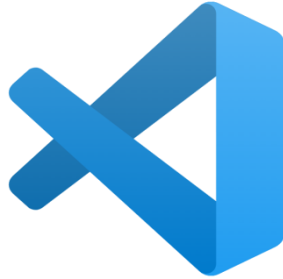


Fig. 1.16 Visual Studio Code

VS Code offers advanced features such as intelligent code completion (IntelliSense), syntax highlighting, and real-time error detection, which significantly improve coding efficiency and accuracy. It also supports a wide range of extensions that enable seamless integration with frameworks like Spring Boot, allowing developers to manage dependencies, configure applications, and streamline development workflows.

VS Code also offers built-in support for version control systems such as Git, enabling developers to perform operations like committing, pushing, and pulling code directly within the environment. This enhances collaboration and simplifies code management.

Usage of Visual Studio Code in the Project

Visual Studio Code was used as a primary development environment for writing, editing, and managing frontend and backend components of the application. It facilitated efficient code development, debugging, and integration with version control systems, contributing to a streamlined and productive development process.

3. Java (Backend)

Java is a high-level, object-oriented programming language that is widely used for developing web applications, enterprise software, and distributed systems. It was developed by Sun Microsystems and is now maintained by Oracle Corporation. Java is known for its platform independence, which means that programs written in Java can run on any system that has a Java Virtual Machine (JVM). This feature is commonly described as “*Write Once, Run Anywhere.*”

One of the major advantages of Java is its robustness and reliability. It provides features such as automatic memory management through garbage collection, exception handling, and strong type checking, which help in reducing errors and improving program stability.

Java is also widely used in backend development, especially with frameworks like Spring Boot. In this project, Java is used to develop microservices that handle different functionalities such as user management, car listing, bidding, and payment processing. Each microservice is built using Java and communicates with other services through APIs.



Fig. 1.17 Java

Java follows the principles of object-oriented programming (OOP), which include concepts such as classes, objects, inheritance, polymorphism, encapsulation, and abstraction. Another important feature of Java is its security. It provides built-in security mechanisms such as bytecode verification, secure class loading, and runtime security checks, making it suitable for developing secure applications.

In conclusion, Java is a powerful and versatile programming language that plays a crucial role in developing scalable, secure, and efficient applications. Its compatibility with modern technologies like microservices and cloud platforms makes it an ideal choice for backend development in real-world projects.

4. React (Frontend)

React JS is a popular JavaScript library used for building user interfaces, especially for single-page applications. It was developed by Facebook and is widely used for creating dynamic and interactive web applications. React follows a component-based architecture, where the user interface is divided into small reusable components. Each component represents a part of the UI, such as a navigation bar, form, or card. This makes the application easy to manage, maintain, and scale.

One of the key features of React is the use of **Virtual DOM**, which improves performance by updating only the required parts of the web page instead of reloading the entire page. This

makes applications faster and more efficient. React also uses **JSX (JavaScript XML)**, which allows developers to write HTML-like code inside JavaScript. This makes the code more readable and easier to understand.

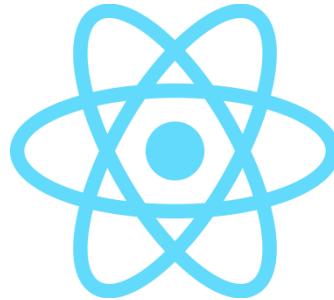


Fig. 1.18 React JS

In this project, React JS is used to develop the frontend of the C2C car resale platform. It is used to design user interfaces such as login page, car listing page, search and filter functionality, and user interaction components.

React also supports state management and routing, which helps in handling user data and navigation between different pages. In conclusion, React JS is a powerful and efficient frontend technology that helps in building responsive and user-friendly web applications.

5. MySQL Workbench

Introduction

MySQL Workbench is a comprehensive integrated development environment (IDE) developed by Oracle for designing, developing, and administering MySQL databases. The tool simplifies complex database operations through its graphical interface and advanced features, making it widely used by database administrators, developers, and students. The tool provides a visual interface for designing Entity-Relationship (ER) diagrams, which helped in planning and organizing the database architecture efficiently.



Fig. 1.19 MySQL Workbench

It was primarily utilized for creating and managing database schemas, tables, and relationships, ensuring proper structuring of data according to application requirements.

MySQL Workbench was also used to execute SQL queries for performing operations such as data insertion, retrieval, updating, and deletion. This enabled efficient handling of application data and testing of database functionality during development.

The tool also supports data modeling and forward/reverse engineering, allowing synchronization between database design and actual implementation. Overall, MySQL Workbench played a vital role in managing the database layer of the project, ensuring efficient data handling and reliable system performance.

Usage of MySQL Workbench in the Project

MySQL Workbench was used to design and manage the database, execute SQL queries, and handle data storage for the application. It also facilitated the creation of database schemas, table relationships, and efficient data retrieval, ensuring proper organization and integrity of the data.

6. Jenkins

Introduction

Jenkins is an open-source automation tool used in software development for building, testing, and deploying applications. It plays a crucial role in DevOps by enabling Continuous Integration (CI) and Continuous Deployment (CD). Jenkins automates repetitive tasks, improves development efficiency, and ensures faster delivery of software.



Fig. 1.20 Jenkins

The tool was used to create and manage CI/CD pipelines, which included stages such as code checkout, project build (using Maven), running test cases, and performing code quality analysis using tools like SonarQube. This automated workflow helped in identifying errors early in the development cycle and maintaining code quality.

Jenkins was configured to integrate with the version control system (GitHub), enabling automatic triggering of pipelines whenever code changes were committed. This ensured that every code update was continuously integrated into the main project and validated through automated processes.

Additionally, Jenkins facilitated continuous deployment, allowing the application to be deployed automatically to the local or target environment after successful build and testing. This reduced the time required for manual deployment and ensured consistency in the release process. Jenkins also provides extensive plugin support, enabling integration with various tools and technologies used in the project. This improves transparency and makes the workflow easier to manage and modify.

Usage of Jenkins in the Project

Jenkins was used to automate the CI/CD pipeline, including building the project, running tests, and deploying the application after successful code commits.

7. SonarQube

Introduction

SonarQube is an open-source tool used for continuous inspection of source code quality. It helps in identifying bugs, vulnerabilities, and code smells, thereby improving the quality, security, and maintainability of software. It can be integrated with CI/CD pipelines for automated code analysis.



Fig. 1.21 SonarQube

It helps in identifying code smells, which are not necessarily bugs but indicate poor coding practices that may affect code readability and maintainability in the long run.

Additionally, SonarQube enforces quality gates, which define specific conditions that the code must meet before being accepted.. SonarQube supports multiple programming languages such as Java, Python, JavaScript, and more, making it a versatile tool for analyzing different types of projects.

The tool was integrated with Jenkins in the CI/CD pipeline, enabling automated code analysis after each build. This ensured that every code change was evaluated for quality standards before deployment.

This helps developers prioritize improvements effectively. Overall, SonarQube played a crucial role in enhancing code quality, ensuring security, and promoting best coding practices throughout the project development lifecycle.

Usage of SonarQube in the Project

SonarQube was integrated with Jenkins to perform automated code quality checks, detect bugs and vulnerabilities, and enforce quality gates before deployment.

8. GitHub

Introduction

GitHub is a web-based platform used for version control and collaboration. It is built on Git and allows developers to store, manage, and track changes in their source code.

It is widely used in software development for team collaboration and project management. GitHub supports branching and merging, which enables multiple developers to work on different features or modules simultaneously without affecting the main codebase. Once development is complete, changes can be merged into the main branch in a controlled manner.



Fig. 1.22 GitHub

It allows developers to maintain a centralized repository where all project files and code are stored. Each change made to the codebase is tracked using commits, ensuring that the complete history of modifications is preserved. This helps in maintaining transparency and allows developers to revert to previous versions if required. In addition, GitHub was integrated with tools like Jenkins and SonarQube in the CI/CD pipeline, allowing automated build, testing, and code quality analysis whenever changes were pushed to the repository.

GitHub ensured effective version control, collaboration, and seamless integration with the development and deployment pipeline. Overall, GitHub played a vital role in managing version control, enabling team collaboration, and ensuring a structured and efficient development workflow.

9. Git

Introduction

Git is a distributed version control system used to track changes in source code during software development. It allows developers to manage code versions efficiently and collaborate on projects. It allows developers to save changes in the form of commits, which helps in maintaining a complete history of the project. Git supports branching and merging, which allows developers to work on new features or bug fixes independently without affecting the main codebase. Once the work is completed, the changes can be merged into the main branch in a controlled manner.



Fig. 1.23 git

Overall, Git played a crucial role in maintaining code integrity, improving collaboration, and managing the project efficiently.

Common Git Commands

```
git init # Initialize a new Git repository
git clone <url> # Clone a repository from GitHub
git status # Check current status of files
git add . # Add all changes to staging area
git commit -m "msg" # Save changes with a message
git push origin main # Push code to GitHub repository
git pull origin main # Fetch and merge latest changes
git branch # List all branches
git checkout <branch> # Switch to another branch
git merge <branch> # Merge branch into current branch
```

Usage of Git in the Project

Git was used for version control to track changes in the source code, maintain different versions of the project, and manage code updates efficiently during development.

10. Docker

Introduction

Docker is an open-source platform used to develop, ship, and run applications inside lightweight containers. Docker is a containerization tool that allows applications to run in isolated environments called containers. These containers include everything needed to run the application such as code, libraries, and system tools.

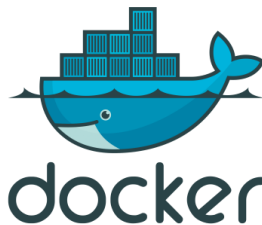


Fig. 1.24 Docker

Docker allows applications to be bundled along with all their dependencies, including libraries, configurations, and runtime environments, into a single container. This eliminates environment-related issues, commonly known as the “it works on my machine” problem, and ensures uniform behavior across systems.

Docker was used to create container images for different microservices, enabling each service to run independently in its own isolated environment. It integrates well with CI/CD tools like Jenkins, enabling automated build and deployment pipelines.

Basic Docker Commands

```
docker --version    # Check Docker version
docker pull nginx    # Download image
docker run nginx     # Run container
docker ps           # List running containers
docker stop <id>     # Stop container
docker build -t app . # Build image
```

Usage of Docker in the Project Docker was used in the project for deployment purposes by containerizing each microservice along with its required dependencies, configurations, and runtime environment. This approach ensured that every service runs in an isolated container, eliminating issues related to environment differences between development, testing, and production.

By using Docker, the application could be packaged into lightweight and portable containers, making it easier to deploy and manage across different systems.

11. Claude

Introduction

Claude is an advanced AI-based assistant developed by Anthropic, designed to support software development and problem-solving tasks. In this project, Claude was used as an intelligent assistant to enhance productivity and assist in various stages of development.



Fig. 1.25 Claude

It was primarily used for generating code snippets, suggesting solutions, and debugging errors, which helped in accelerating the development process. Claude also assisted in understanding complex concepts, designing system components, and improving code quality by providing optimized approaches and best practices. Claude is capable of performing tasks such as answering queries, generating text, writing code, debugging programs, and explaining complex concepts in a simplified manner.

One of the key purposes of Claude is to enhance productivity by providing quick and accurate assistance in various domains, including software development, education, and content creation. It can analyze user input, understand context, and provide relevant suggestions or solutions efficiently. The tool also helped in API design, microservices development guidance, and troubleshooting issues, enabling faster problem resolution and better decision-making during implementation.

Claude is also widely used for documentation, report writing, and research support, as it helps in organizing information and presenting it in a structured and professional format. This made it easier to maintain clear and professional project documentation. Claude enhances productivity by providing intelligent assistance in coding, content creation, and problem-solving tasks.

Usage of Claude in the Project

Claude was used to create the initial project prototype, generate code snippets, and assist in understanding and implementing various components of the microservices architecture.

12. Postman

Introduction

Postman is a widely used API development and testing tool designed to simplify the process of building, testing, and debugging APIs. In this project, Postman was used as an essential tool to test and validate the functionality of REST APIs developed for the system.



Fig. 1.26 Postman

Postman provides a user-friendly interface that allows developers to send HTTP requests and analyze responses easily. It supports various request methods such as GET, POST, PUT, and DELETE, which are commonly used in RESTful services. In modern software development, especially in microservices architecture, APIs play a crucial role in communication between different services. Postman helps in ensuring that these APIs are working correctly by verifying request parameters, response data, and status codes.

Usage of Postman in the Project

In the MarutiXchange project, Postman was used extensively for API testing and validation.

It was used for:

- Testing user authentication APIs (login and registration)
- Verifying car listing and search APIs
- Testing booking and bidding functionalities.

PART -2 PROJECT

MICROSERVICES-BASED FULL-STACK C2C CAR RESALE PLATFORM

(marutiXchange)

CHAPTER -2 PROJECT DETAILS

2.1 INTRODUCTION TO THE PROJECT

The project titled “**MarutiXChange**” is designed to provide a digital marketplace that enables direct interaction between buyers and sellers for the resale of cars. The platform aims to eliminate intermediaries and bring transparency, efficiency, and flexibility into the car resale process.

2.1.1 Problem Statement

The used car resale market in India is highly fragmented and lacks a unified platform that ensures transparency, efficiency, and direct customer interaction. Existing solutions either rely heavily on intermediaries or do not provide a structured and reliable mechanism for transactions.

Manufacturer-backed platforms such as those by Maruti Suzuki operate through dealer networks, where cars are inspected, refurbished, and sold via authorized dealers. While these platforms ensure quality and trust, they do not support direct customer-to-customer (C2C) interaction, limiting flexibility and potentially increasing costs due to intermediary involvement.

On the other hand, general-purpose classified platforms like OLX and Quikr enable direct interaction between buyers and sellers. However, these platforms are not brand-specific, lack standardized processes, and do not provide advanced features such as structured bidding systems, dynamic pricing, or rule-based validation mechanisms.

Furthermore, hybrid platforms such as Cars24 and Spinny act as intermediaries, handling inspection, pricing, and transactions, thereby reducing direct user control and interaction.

There is currently no dedicated platform that combines:

- Direct C2C interaction
- Brand-specific focus (Maruti vehicles only)
- Real-time bidding mechanisms
- Rule-based pricing and validation

This gap highlights the need for a specialized, scalable, and transparent system that enables direct transactions between customers while maintaining structured workflows and industry-level reliability.

2.1.2 Objectives of the Project

The primary objective of this project is to design and develop a **Customer-to-Customer (C2C) car resale platform** specifically for Maruti vehicles, addressing the limitations of existing systems.

The key objectives include:

- To develop a platform that enables direct interaction between buyers and **sellers** without intermediaries
- To provide a brand-specific marketplace focused exclusively on Maruti vehicles
- To implement a real-time bidding system for dynamic price discovery
- To integrate a rule engine for enforcing business logic such as pricing, bidding validation, and booking conditions
- To ensure transparency, efficiency, and reliability in the car resale process
- To design a scalable microservices-based architecture for better maintainability and performance

2.1.3 Proposed Solution

To address the identified challenges, the proposed system introduces a **C2C Car Resale Platform (MarutiXchange)** that enables direct transactions between buyers and sellers while incorporating advanced technological features.

The system provides:

- A user-friendly interface for listing and browsing cars
- Advanced search and filtering capabilities
- A real-time bidding system for competitive price discovery
- Rule-based validation using a rules engine to ensure fairness and correctness in transactions
- Secure booking and transaction handling
- A microservices-based architecture to ensure scalability, flexibility, and independent deployment of services

This solution not only addresses the limitations of existing platforms but also introduces innovative features that enhance transparency, efficiency, and user satisfaction.

2.1.4 System Architecture

The system is designed using a **Microservices Architecture**, where the application is divided into multiple independent and loosely coupled services. Each service is responsible for specific business functionality and communicates with other services through RESTful APIs.

This architectural approach enables better scalability, flexibility, and maintainability compared to traditional monolithic systems. Each microservice can be developed, deployed, and scaled independently without affecting other components of the system.

Architecture Components

The system consists of the following major components:

1. Client Layer

The client layer includes the user interface through which users interact with the system. It is developed using modern frontend technologies and provides features such as car browsing, bidding, and booking.

2. API Layer

The API layer acts as a communication bridge between the client and backend services. It handles incoming requests and routes them to the appropriate microservices.

3. Microservices Layer

The application is divided into multiple microservices, each handling a specific functionality:

- **User Service:** Manages user registration, authentication, and profiles
- **Car Listing Service:** Handles car details and listings
- **Bidding Service:** Manages bidding logic and bid validation
- **Booking Service:** Handles car booking and availability
- **Payment Service:** Manages transaction processing

4. Data Layer

Each microservice maintains its own database to ensure data isolation and independence. MySQL is used as the database system for storing application data.

5. Rule Engine

A rule engine is integrated into the system to handle business logic dynamically. It is used for pricing rules, bidding validation, and booking conditions, allowing flexibility without modifying core application code.

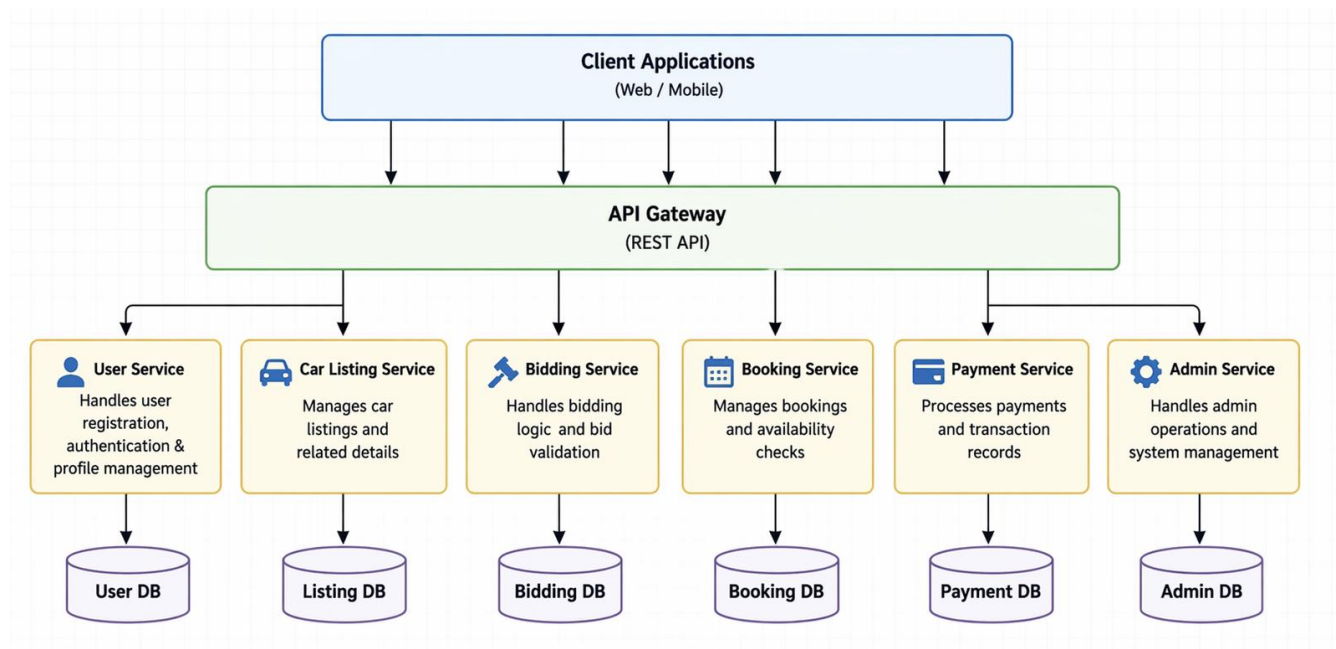


Fig. 2.1 Microservices architecture for C2C Car Resale Platform

2.1.5 Project setup and initialization

The development of the MarutiXchange platform was initiated using Spring Boot, which provides a structured and efficient way to build production-ready applications. Each microservice within the system was created as an independent Spring Boot project using Spring Initializr.

Spring Initializr was used to generate the base project structure with required dependencies. The following dependencies were included based on system requirements:

- Spring Web – for building RESTful APIs
- Spring Data JPA – for database interaction
- MySQL Driver – for database connectivity
- Lombok – for reducing boilerplate code

Each microservice was configured independently with its own application properties and database connection, ensuring loose coupling and modular development.

The project followed a microservices approach where separate services such as User Service, Car Listing Service, Bidding Service, and Booking Service were developed and deployed independently. This setup allowed better scalability, maintainability, and flexibility in system development.

spring initializr

Project
☐ Gradle - Groovy ☐ Gradle - Kotlin ☒ **Java** ☐ Kotlin ☐ Groovy
☒ **Maven**

Spring Boot
☐ 4.1.0 (SNAPSHOT) ☐ 4.1.0 (M4) ☐ 4.0.6 (SNAPSHOT) ☒ **4.0.5**
☐ 3.5.14 (SNAPSHOT) ☐ 3.5.13

Project Metadata
 Group
 Artifact
 Package name
 Packaging ☒ **Jar** ☐ War
 Configuration ☒ **Properties** ☐ YAML
 Java ☐ 26 ☐ 25 ☒ **21** ☐ 17

Dependencies ADD DEPENDENCIES... CTRL + B

Spring Data JPA SQL
 Persist data in SQL stores with Java Persistence API using Spring Data and Hibernate.

Lombok DEVELOPER TOOLS
 Java annotation library which helps to reduce boilerplate code.

Spring Web WEB
 Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

MySQL Driver SQL
 MySQL JDBC driver.

GENERATE CTRL + G EXPLORE CTRL + SPACE ...

Fig. 2.2 Microservices initialization using Spring Initializr for the User Service

The above configuration represents the initialization of a microservice using Spring Initializr. Similar configurations were used to create multiple services such as User Service, Car Listing Service, Bidding Service, and Booking Service.

2.1.6 Module Description

The MarutiXchange platform is divided into multiple functional modules, each responsible for a specific aspect of the system. This modular design ensures scalability, maintainability, and efficient handling of system functionalities.

1. User Module

This module handles user-related operations such as registration, authentication, and profile management. It ensures secure access to the platform and maintains user data.

2. Car Listing Module

This module allows users to list cars for sale by providing details such as model, price, year, and specifications. It also enables retrieval of car listings for buyers.

3. Search and Filter Module

This module provides advanced search functionality, allowing users to filter cars based on parameters such as price range, model, and year.

4. Bidding Module

This module enables users to place bids on listed cars. It ensures that each new bid is higher than the previous bid and validates bidding rules.

5. Booking Module

This module handles the booking process and ensures that cars can only be booked if they are available and meet all conditions.

6. Payment Module

This module manages transaction processing and ensures secure payment handling.

2.1.7 Prototype

A prototype is an initial version of the system that is created to visualize the user interface and understand how the application will function before full development. It helps in representing the layout, design, and flow of the system in a simplified manner.

The main purpose of creating a prototype is to provide a clear understanding of the system's functionality, improve user experience, and identify any design issues at an early stage. It allows developers and users to interact with the system design and suggest improvements before implementation.

In general, prototypes can range from simple sketches to fully interactive designs. In this project, a high-fidelity prototype was considered, as it closely represents the actual system interface and user interactions.

Prototype in MarutiXchange Project

In the MarutiXchange system, prototype screens were designed to represent the major functionalities of the application. These screens help in visualizing how users will interact with the platform.

The prototype includes the following key screens:

- **Login / Registration Screen**
Allows users to create an account or log in to the system.
- **Car Listing Screen**
displays available cars with details such as model, price, and specifications.
- **Search and Filter Screen**
Enables users to search and filter cars based on various criteria such as price, model, and year.

- **Bidding Screen**
Allows users to place bids on cars and view current highest bids.
- **Booking Screen**
Enables users to confirm booking of selected cars.
- **Payment Screen**
Handles payment process and transaction details.

Purpose of Prototype

The prototype was developed to achieve the following objectives:

- To visualize the structure and layout of the system
- To improve the user interface and user experience
- To identify and fix design issues before development

Benefits of Prototype

- Provides a clear idea of system design
- Helps in early feedback and improvements
- Reduces development errors
- Improves usability and user experience

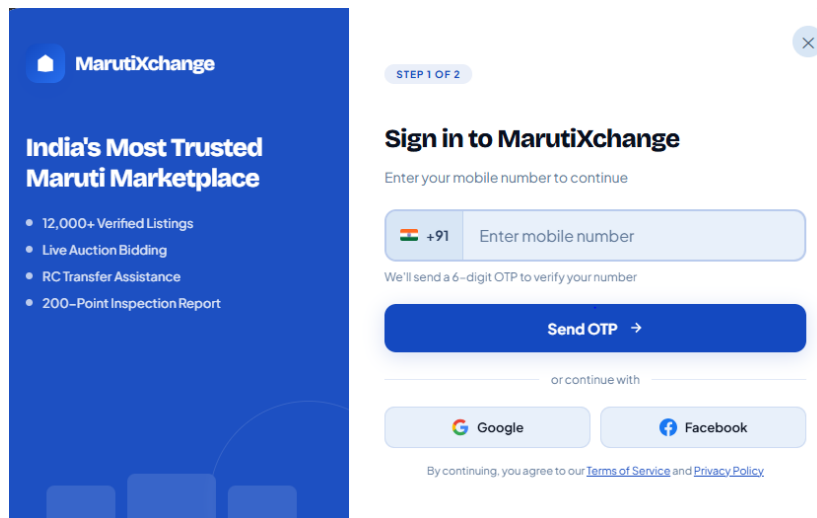


Fig. 2.3 Login Screen

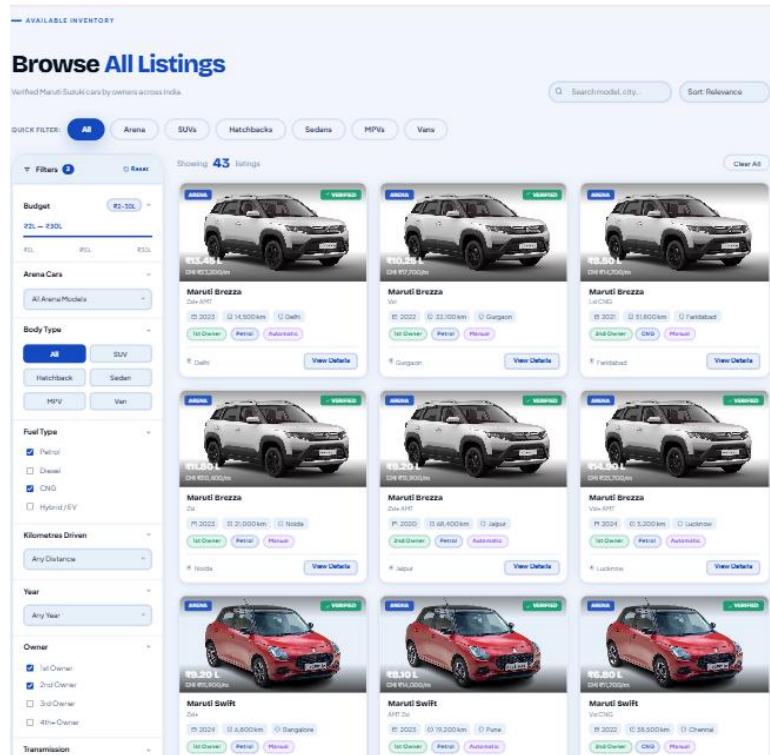


Fig. 2.4 Arena Car Listening Screen

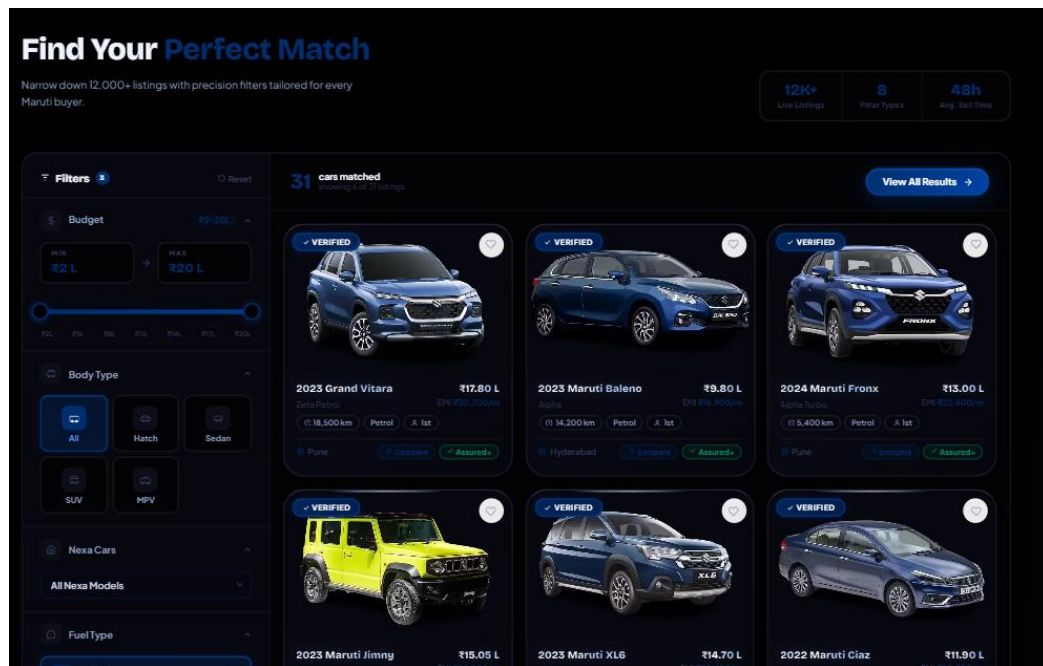


Fig. 2.5 Nexa Car Listening Screen

List Your Car

Complete all steps to receive instant buyer inquiries

₹40,000+

Avg. premium over dealer offers

1

2

3

4

5

Car Info

Photos

Pricing

Test Drive

Contact

Auto-fill via Registration No. (CP-71)

Enter your vehicle number below to auto-fill model, year, fuel & more.

e.g. MH01AB1234

Fetch Details

CAR MODEL *

Select model

VARIANT *

Select model first

FUEL TYPE *

Select fuel

YEAR OF PURCHASE *

Select year

KILOMETRES DRIVEN *

Select range

REGISTRATION NO. *

E.G. DL 01 AB 1234

Continue →

Fig. 2.6 Car Selling Screen

MarutiXchange

ARENA NEKA

Sign In

Search – Brezza, Swift, Baleno...

Buy Cars Sell Car Home Live Bids Dashboard Browse Cars

Bid on Arena Cars

Popular Maruti models – Brezza, Swift, WagonR & more.
Every bid must beat the last. Timer ends, highest wins.

8

LIVE AUCTIONS

20

TOTAL BIDDERS

0

MY ACTIVE BIDS

LIVE



Current Bid

₹6709.53L

23m 22s

Maruti Brezza

2023 23,450 km Petrol Delhi

RESERVE

₹6701.6L

BIDDERS

17

BIDS

33

Verified

Open & Place Bid

LIVE



Current Bid

₹6710.57L

58m 22s

Maruti Swift

2023 12,300 km Petrol Bangalore

RESERVE

₹6701.6L

BIDDERS

17

BIDS

33

Verified

Open & Place Bid

LIVE



Current Bid

₹6711.55L

6m 22s

Maruti Dzire

2021 44,200 km Petrol Chennai

RESERVE

₹6701.6L

BIDDERS

18

BIDS

40

Open & Place Bid

Fig. 2.7 Bidding Car Screen

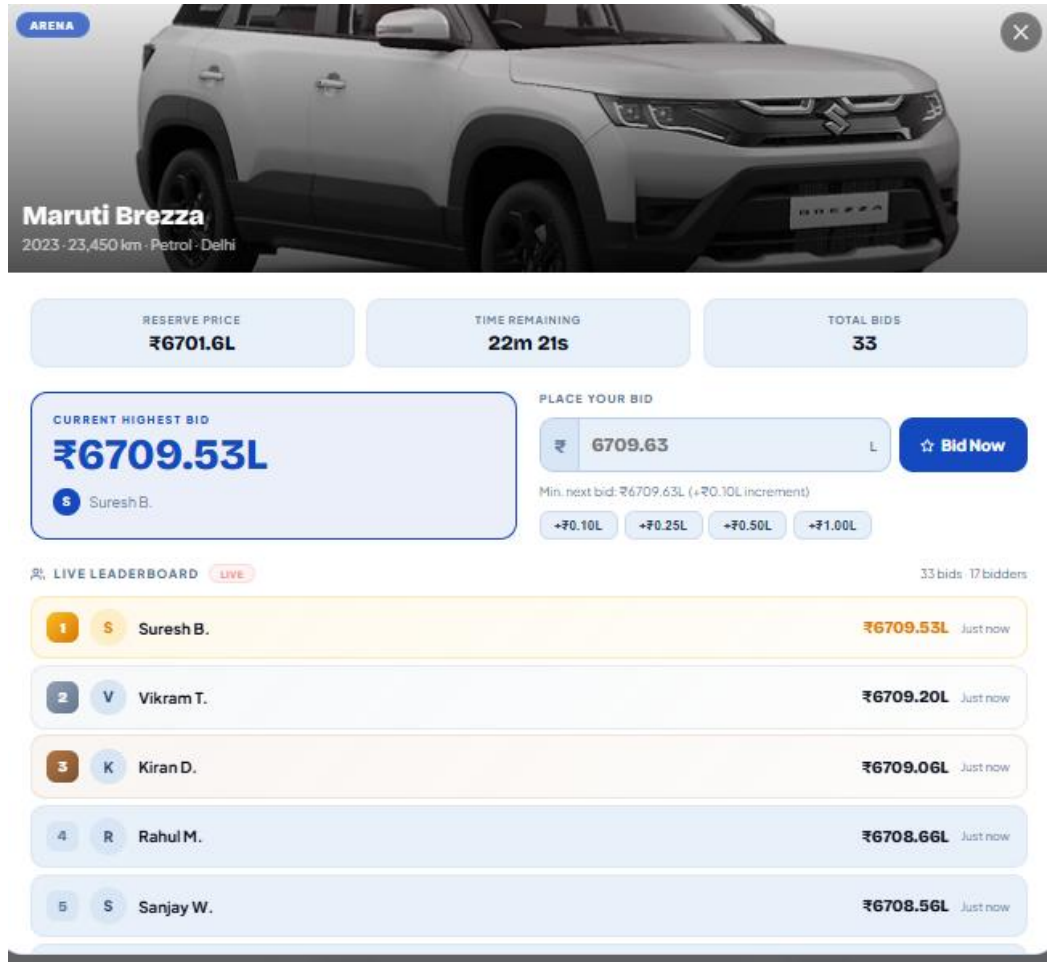


Fig. 2.8 Live Bidding Result Screen

2.1.8 Implementation Details

The implementation of the MarutiXchange system is based on a microservices architecture, where each service is developed as an independent Spring Boot application. These services communicate with each other using RESTful APIs, ensuring loose coupling and scalability.

The system incorporates JWT-based authentication for secure user access, and a rule engine is used to handle dynamic business logic such as bidding validation and pricing rules. Each microservice interacts with its respective database to maintain data independence and integrity.

Execution Path: Bid Processing

Chronological UML Sequence

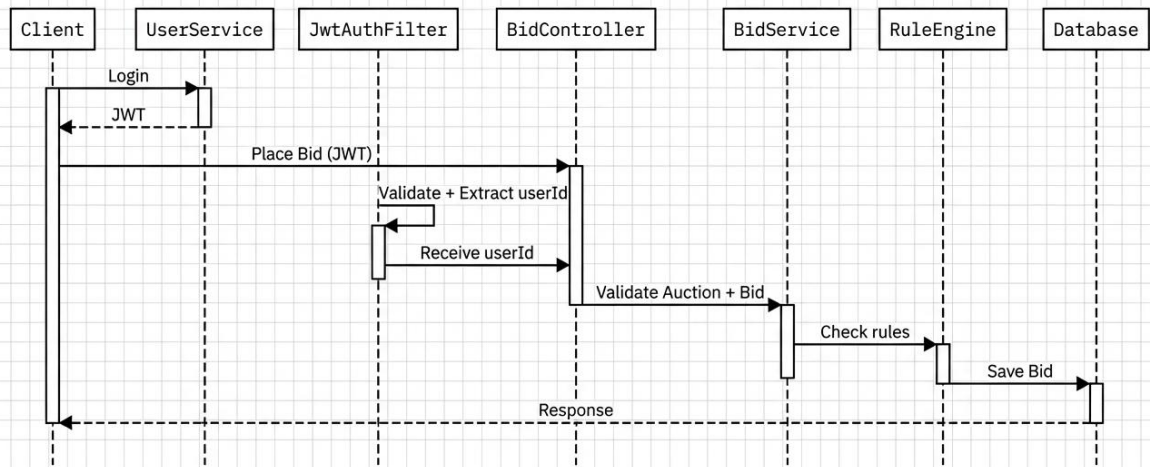


Fig. 2.9 Sequence Diagram of Bid Processing

The sequence diagram illustrates the step-by-step interaction between various system components during the bidding process in the application. It highlights how user requests are authenticated, validated, processed, and stored within the system.

Initially, the user logs into the system by providing valid credentials. Upon successful authentication, the system generates a **JSON Web Token (JWT)**, which is returned to the user. This token is used for subsequent requests to ensure secure communication and authorization.

When the user places a bid on a car, the request is sent from the frontend to the backend **Bid Controller**, along with the JWT token in the request header. Before processing the request, the system applies a **JWT Authentication Filter**, which intercepts the request to validate the token. The filter verifies the token's authenticity, checks its expiration, and extracts the user's identity and roles.

Once the token is successfully validated, the request is forwarded to the **Bid Service**, which contains the core business logic for handling bidding operations. The Bid Service first retrieves the current highest bid for the selected car from the database.

After successfully storing the bid, a confirmation response is generated and sent back through the controller to the frontend. The user is then notified that the bid has been successfully placed. In case of validation failure (e.g., bid amount too low or invalid conditions), an error response is returned to the user with an appropriate message.

This sequence ensures secure, reliable, and rule-based bid processing while maintaining data integrity and system scalability.

2.2 BLOCK DIAGRAM OF THE PROJECT

2.2.1 Block Diagram of Microservices-Based C2C Car Resale Platform

The block diagram represents the overall architecture of the proposed system, which is a microservices-based full-stack car resale platform. The system is designed to enable users to buy and sell cars efficiently through a web-based interface.

The user interacts with the system through the frontend application developed using React.js. The frontend provides a user-friendly interface for performing operations such as registration, login, car listing, searching, chatting, and payment.

All user requests are sent to the API Gateway, which acts as a single entry point to the backend services

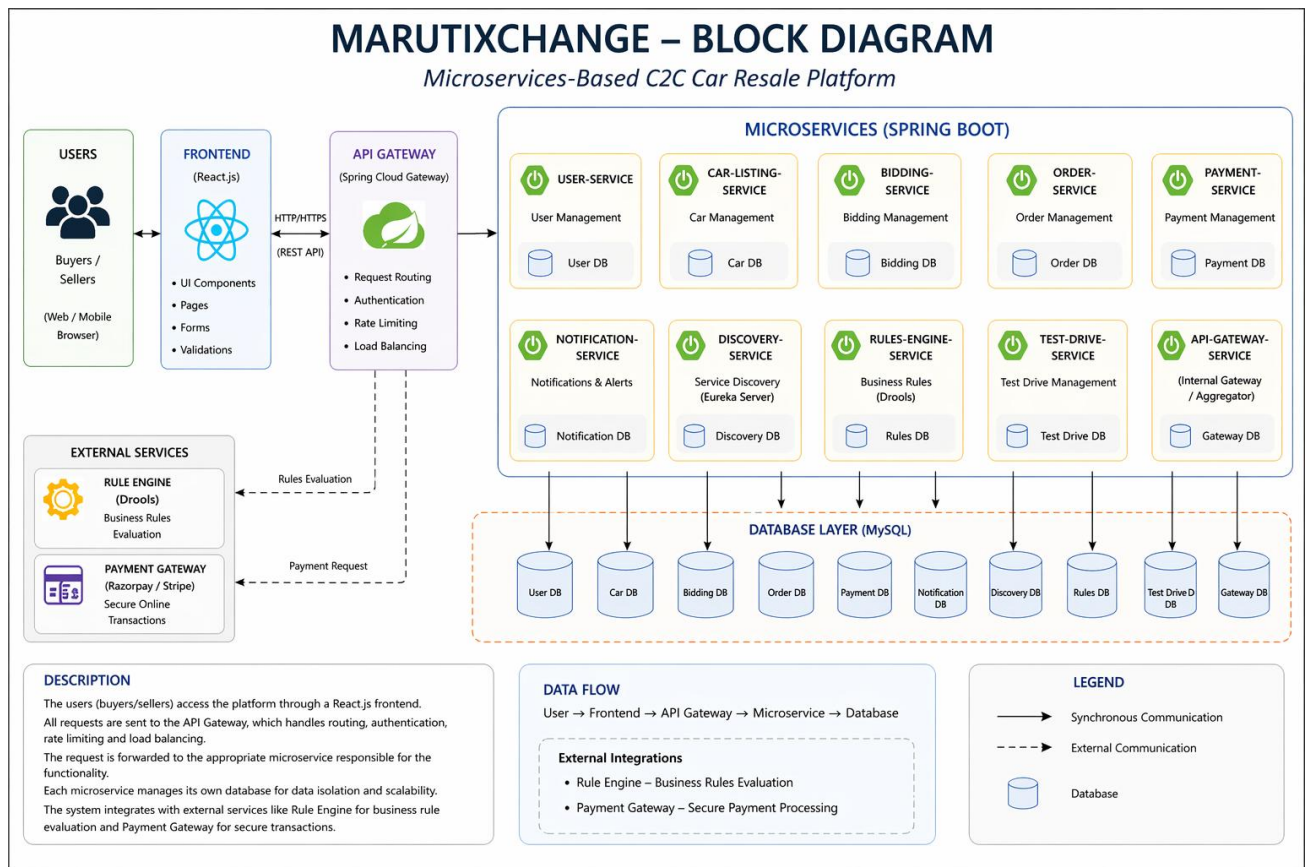


Fig. 2.10 Block Diagram of marutiXchange

2.2.2 Microservices Interaction Diagram

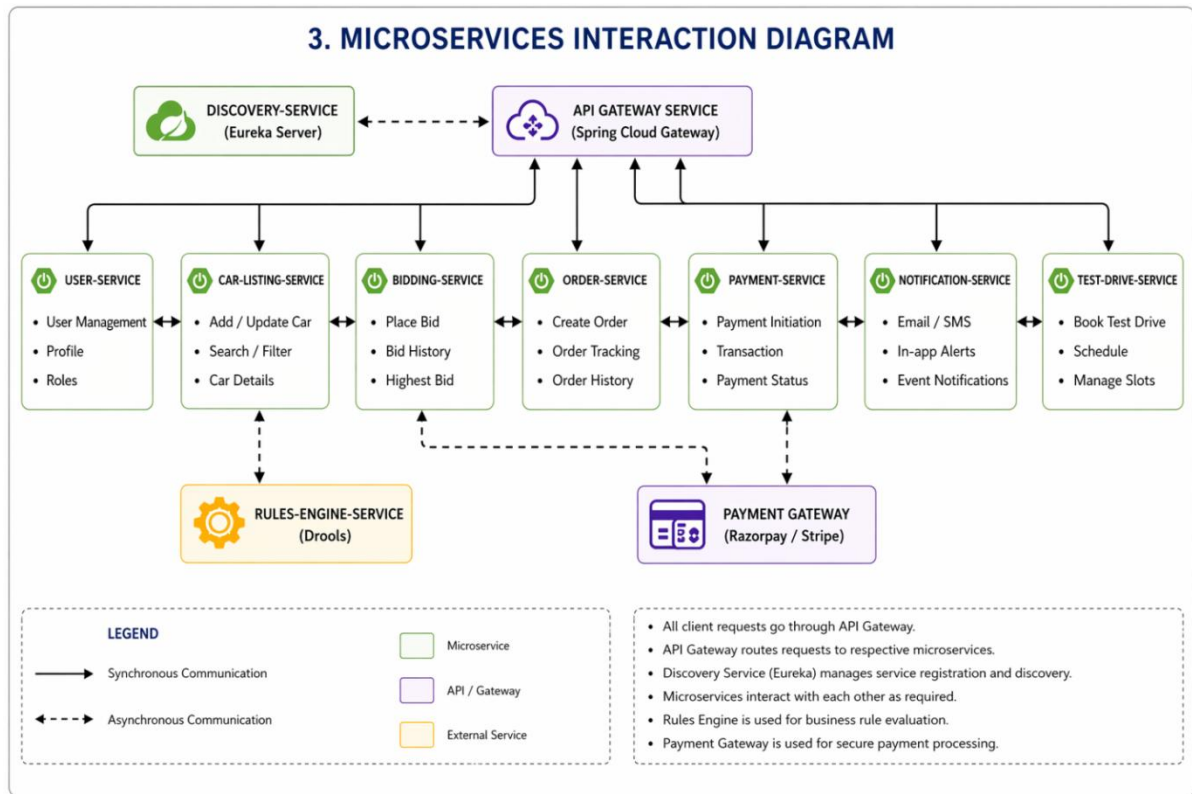


Fig. 2.11 Microservices Interaction Diagram

The above diagram illustrates the interaction between various microservices in the system architecture. Each microservice is designed to handle a specific business functionality:

- The User Service manages user-related operations such as profile and role management.
- The Car Listing Service handles operations related to adding, updating, and searching cars.
- The Bidding Service manages bidding activities including placing bids and tracking bid history.
- The Order Service is responsible for order creation and tracking.
- The Payment Service handles payment initiation, transaction processing, and status updates.
- The Notification Service manages communication through email, SMS, and in-app notifications.
- The Test Drive Service allows users to book and manage test drives.

2.2.3 Database ER Diagram

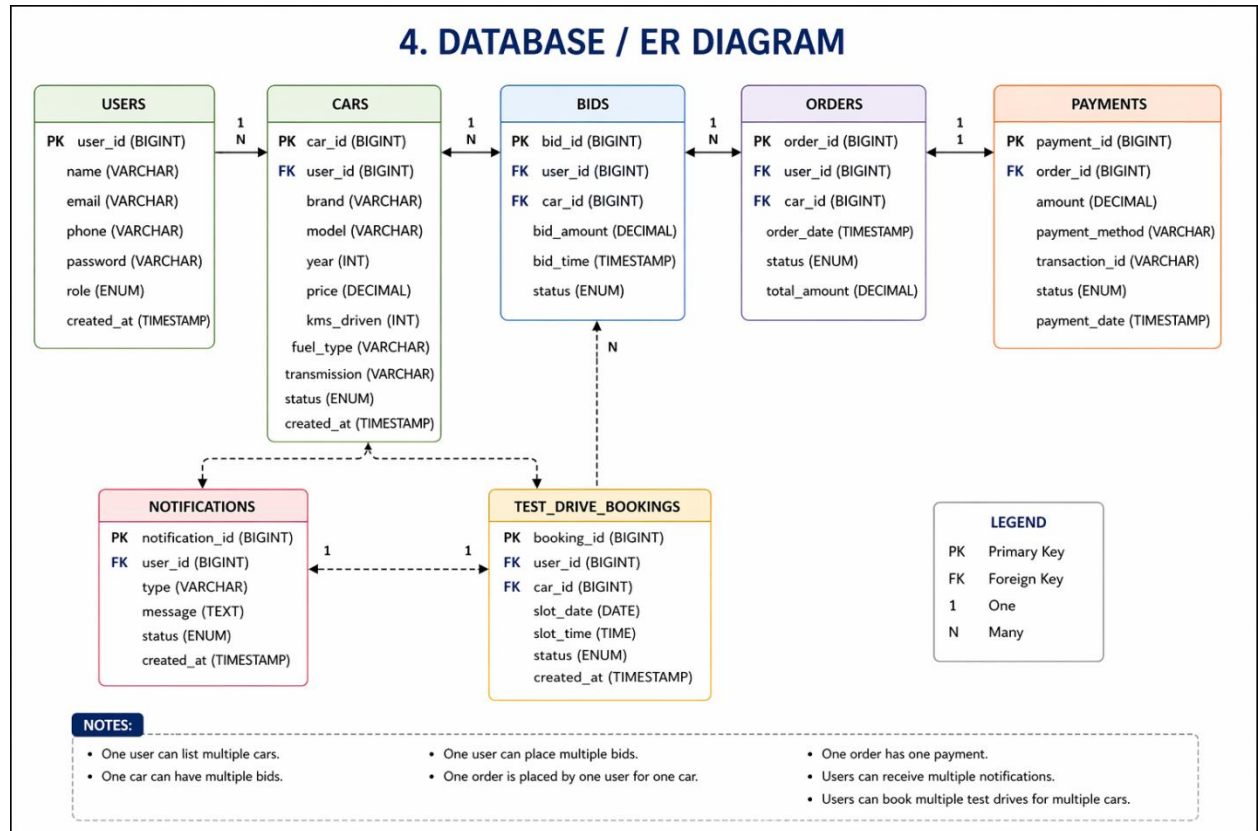


Fig. 2.12 Database (ER) Diagram

The above ER diagram represents the database structure of the system. It shows the main entities such as Users, Cars, Bids, Orders, Payments, Notifications, and Test Drive Bookings along with their attributes and relationships.

Each entity is connected through primary and foreign keys, ensuring data consistency and integrity. The relationships define how users interact with cars, place bids, create orders, and make payments. The diagram also illustrates additional features such as notifications and test drive bookings.

This design supports efficient data management and ensures proper organization of system data.

2.3 FLOWCHART OF THE PROJECT

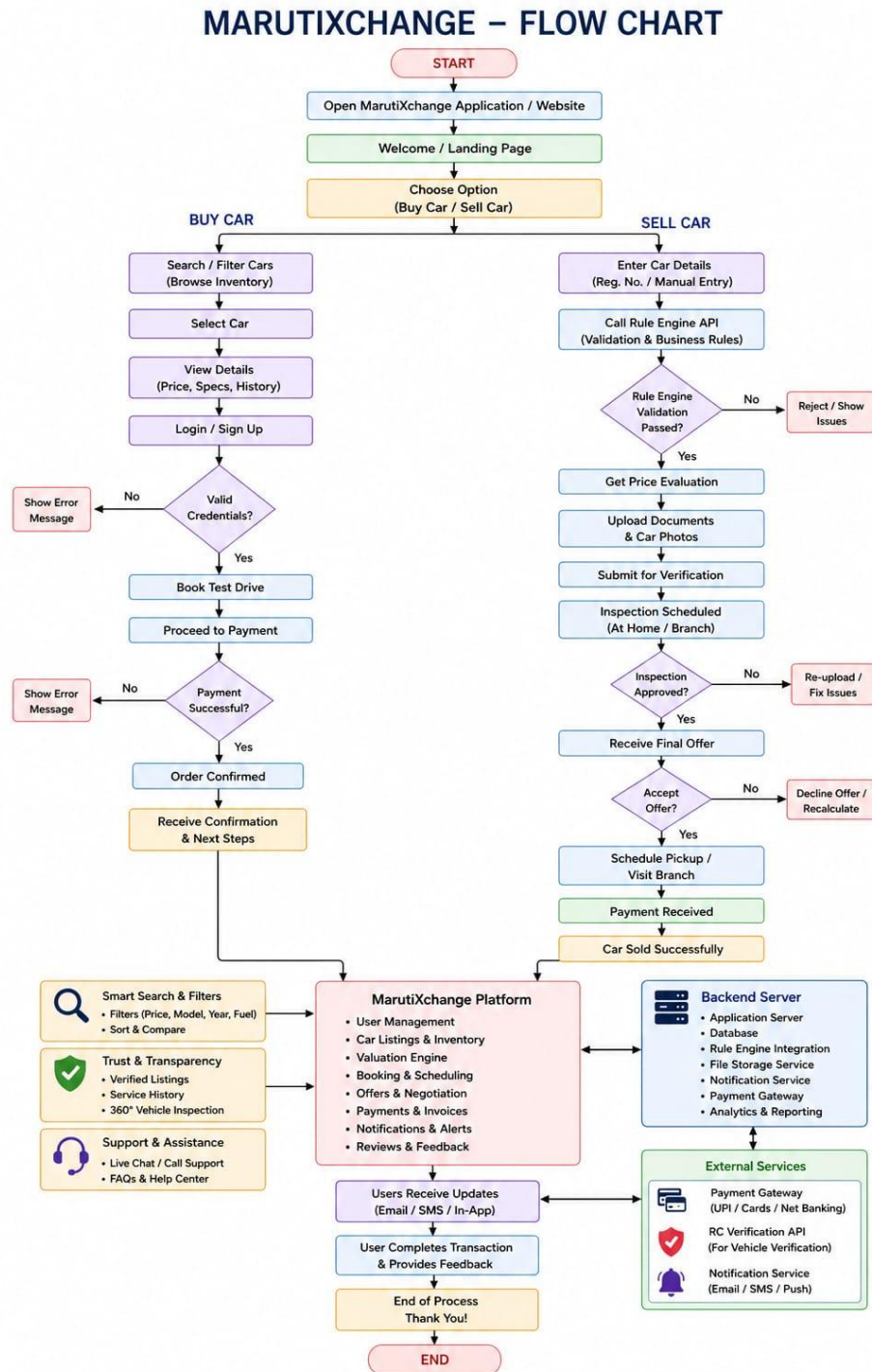


Fig. 2.13 Flow Chart of the Project

2.3.1 DFD LEVEL 0

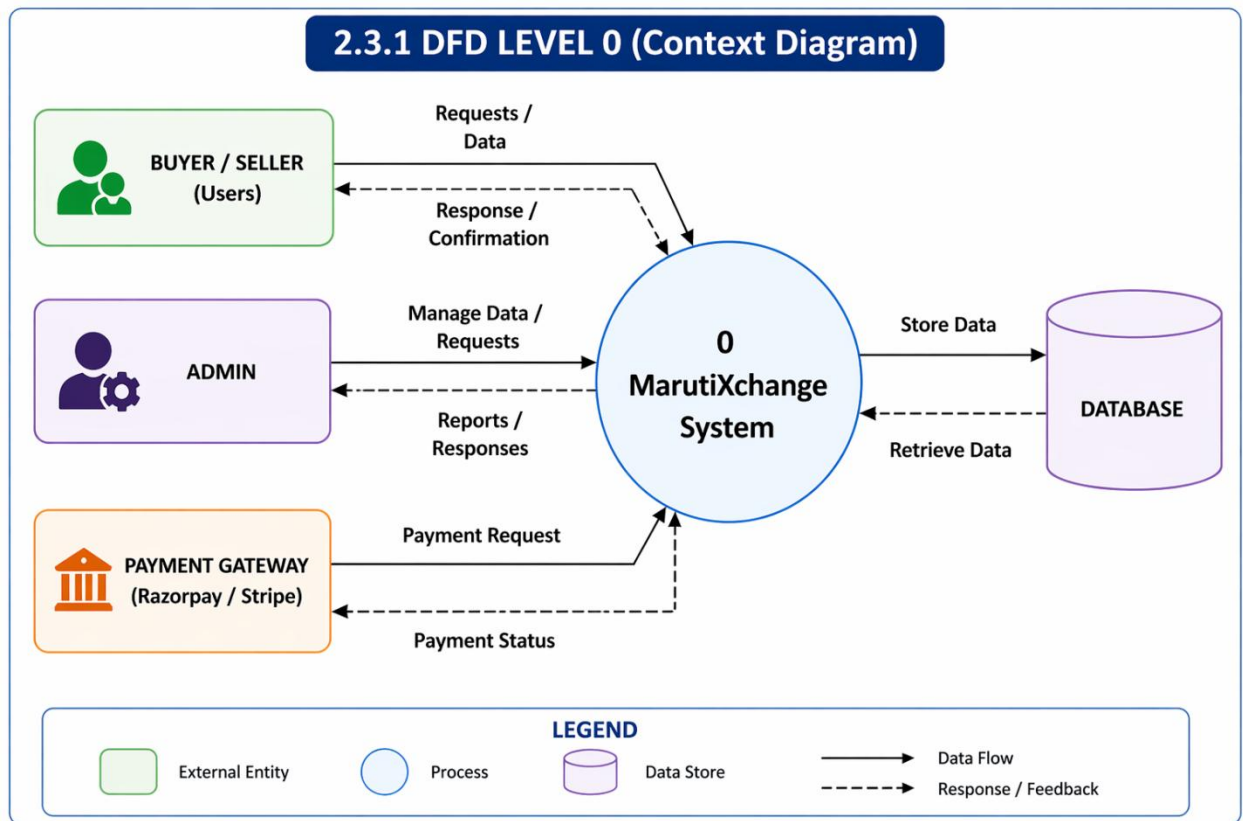


Fig. 2.14 DFD Level 0 – Context Diagram of MarutiXchange System

The DFD Level 0, also known as the context diagram, provides a high-level overview of the entire system. It shows how the system interacts with external entities such as Buyer/Seller, Admin, and Payment Gateway.

The Buyer/Seller interacts with the system to perform activities such as viewing car listings, placing bids, and making payments. The Admin manages system data, monitors activities, and receives reports from the system. The system communicates with the Payment Gateway to process online transactions and receives payment status updates.

All the data generated during these operations is stored in a centralized database, and required information is retrieved whenever needed. This diagram does not show internal processing details but focuses on overall system interaction.

2.3.2 DFD LEVEL 1

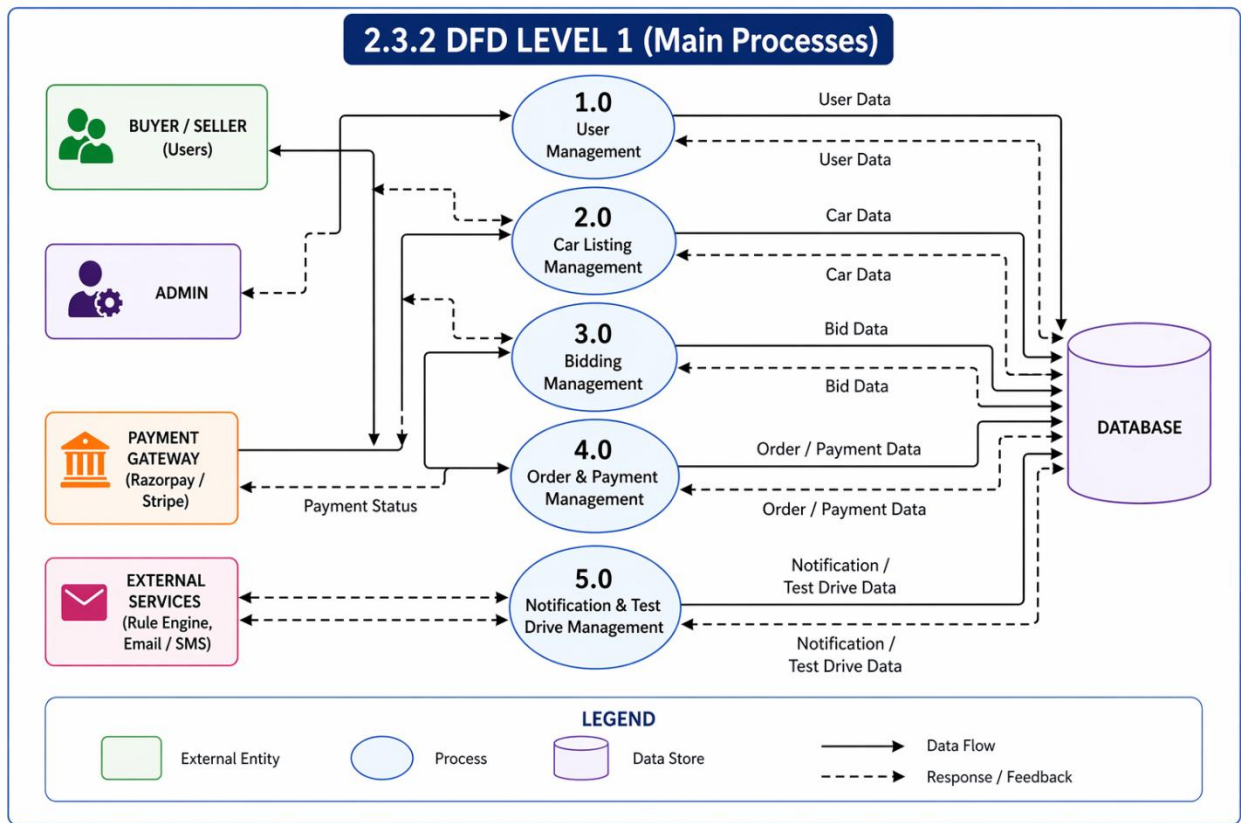


Fig. 2.15 DFD Level 1 – Main Processes of the System

The DFD Level 1 breaks down the system into its main functional processes. It shows how data flows between different processes and the database.

The system consists of several core processes:

- **User Management (1.0):** Handles user registration, login, and profile management.
- **Car Listing Management (2.0):** Manages car details such as adding, updating, and searching listings.
- **Bidding Management (3.0):** Handles bid placement, validation, and bid tracking.
- **Order & Payment Management (4.0):** Manages order creation and payment transactions.
- **Notification & Test Drive Management (5.0):** Handles notifications and test drive bookings.

2.3.3 DFD LEVEL 2

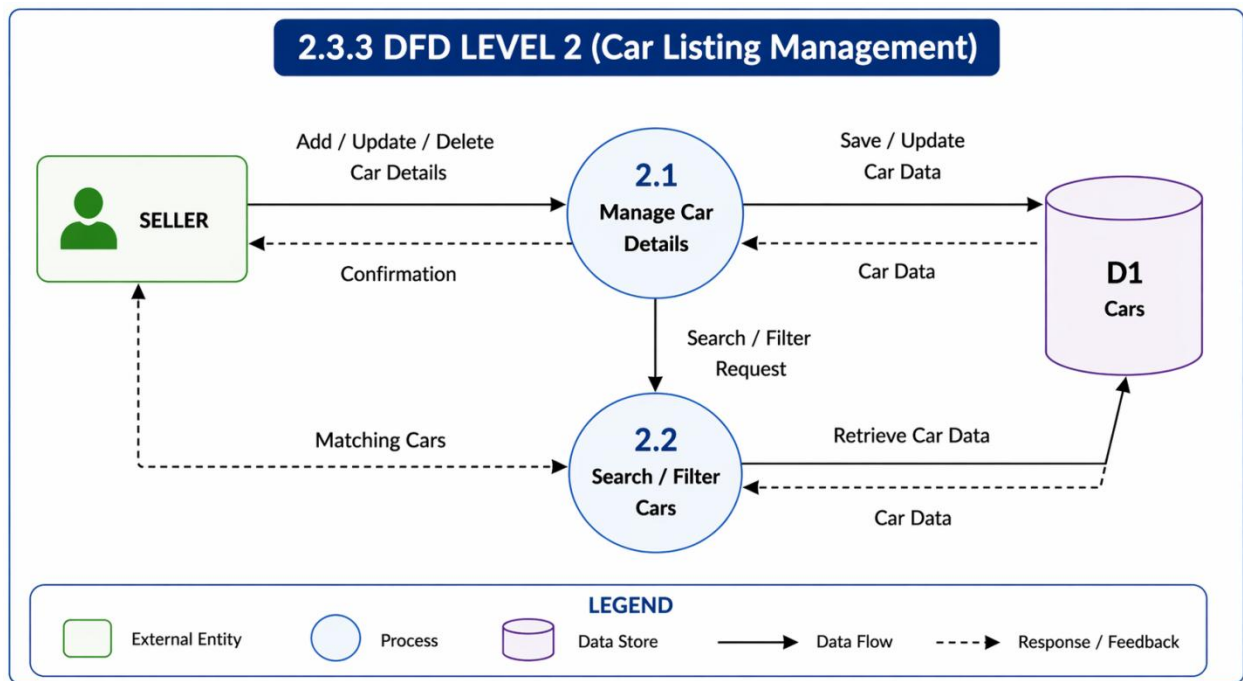


Fig. 2.16 DFD Level 2 – Car Listing Management

The diagram illustrates the detailed workflow of the car listing and management process within the system. It demonstrates how car-related data is handled efficiently through different functional modules.

The **Seller** interacts with the system via the process “**Manage Car Details (2.1)**”, which enables the addition, modification, and deletion of car listings. During this process, the system performs necessary validation checks to ensure the accuracy and consistency of the entered data. These validations may include checking mandatory fields, verifying data formats, and ensuring logical correctness of the information provided. Once validated, the information is stored in the **Cars database** for future access and operations.

Furthermore, the system provides a search and filtering functionality through the process “**Search/Filter Cars (2.2)**”. This feature allows users to retrieve car listings based on specific criteria such as price, model, fuel type, location, and other attributes. The system processes user queries, fetches relevant data from the database, and presents the results in an organized and user-friendly manner, improving accessibility and usability.

Additionally, the system ensures that the stored data is consistently updated and readily available for retrieval, supporting real-time interactions. It may also incorporate sorting mechanisms and advanced filters to refine search results further, enabling users to make more informed decisions.

2.3.4 DFD LEVEL 2

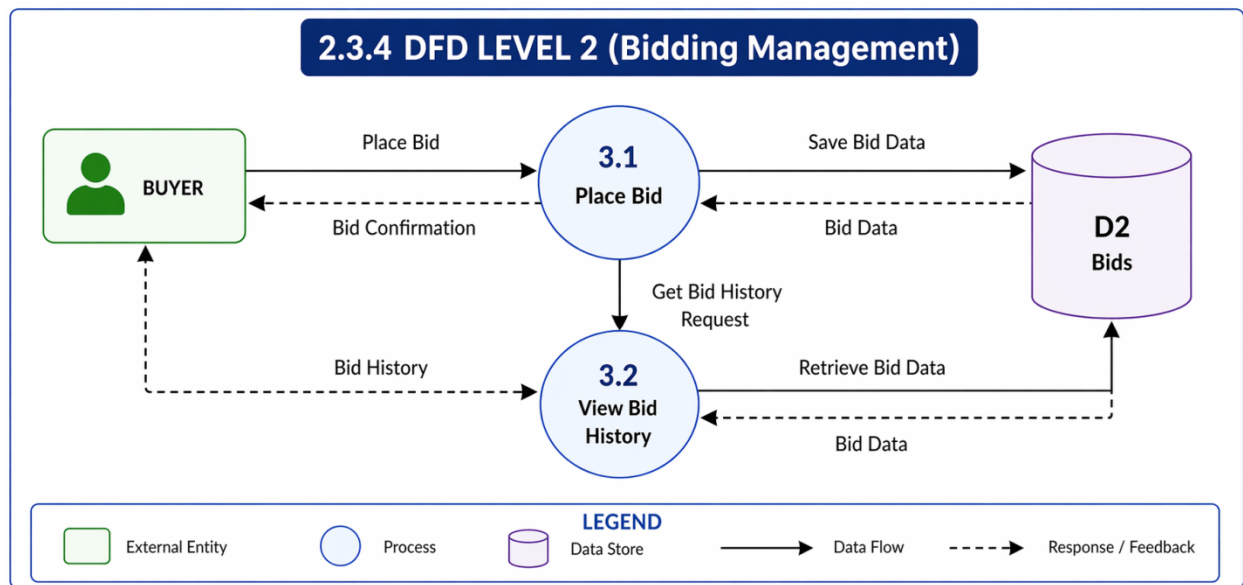


Fig. 2.17 DFD Level 2 –Bidding Management

The diagram illustrates the detailed workflow of the bidding process within the system, highlighting the interaction between the buyer and the system components.

The **Buyer** initiates the process by placing a bid through the process “**Place Bid (3.1)**”. Upon receiving the bid, the system performs validation checks to ensure that the entered bid amount and related details meet the required criteria. Once validated, the bid information is securely stored in the **Bids database**. After successful submission, the system generates and sends a confirmation response to the user, indicating that the bid has been recorded successfully.

In addition, the system provides functionality to track previously placed bids through the process “**View Bid History (3.2)**”. This process retrieves stored bid records from the database and presents them to the user in a structured format, enabling easy monitoring of bidding activities.

The system may also notify users about updates related to their bids, such as bid status or competing bids, enhancing user awareness. Furthermore, proper data handling mechanisms ensure the integrity and security of bid information throughout the process.

Overall, this process ensures transparency, accurate record-keeping and efficient management of bids, thereby providing a smooth and reliable bidding experience for users.

2.4 CODING OF THE PROJECT

The coding of the project is carried out using a microservices-based architecture. The backend is developed using Spring Boot, where multiple independent microservices are created to handle different functionalities such as user management, car listing, bidding, and payment processing. The frontend is developed using React.js, which provides an interactive user interface and communicates with backend services through REST APIs.

2.4.1 Backend Coding

The backend of the system consists of multiple microservices, each responsible for a specific functionality. These services are developed using Spring Boot and follow RESTful architecture. The main services include User Service, Car Listing Service, Bidding Service, and Payment Service.

Backend Microservices

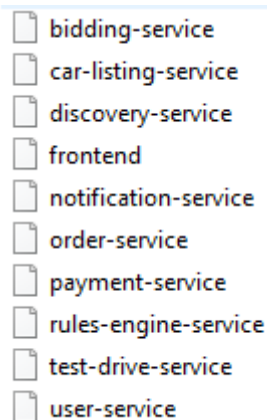


Fig. 2.18 List of Backend Microservices in the System

The above figure shows the different microservices developed in the backend of the system. Each microservice is designed to handle a specific functionality independently.

The system includes services such as user-service for user management, car-listing-service for managing car details, bidding-service for handling bids, order-service for order processing, payment-service for transactions, notification-service for alerts, test-drive-service for scheduling test drives, rules-engine-service for business rule validation, and discovery-service for service registration.

Project Structure

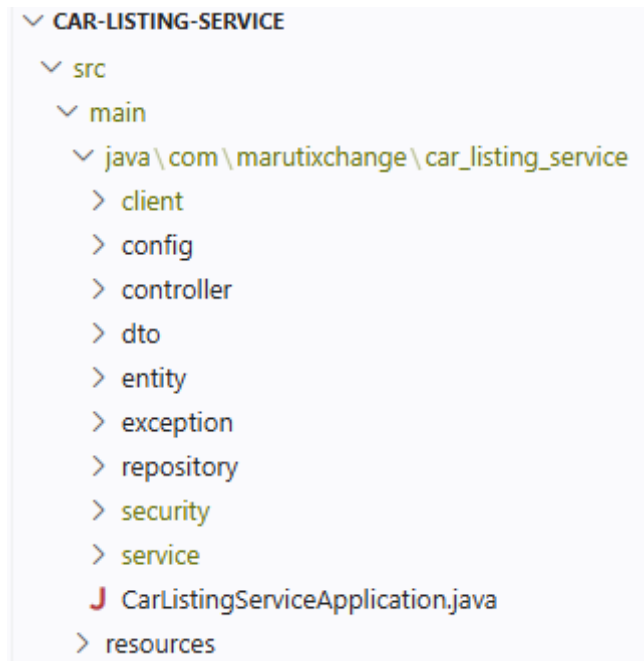


Fig. 2.19 Project Structure of Car Listing Service

Code Implementation

Controller

```
@RestController
@RequestMapping("/api/v1/listings")
@RequiredArgsConstructor
public class CarListingController {

    private final CarListingService listingService;

    @PostMapping
    public ResponseEntity<ApiResponse<CarListingResponse>> createListing(
        @Valid @RequestBody CarListingRequest request) {

        CarListingResponse response =
            listingService.createListing(request);

        return ResponseEntity
            .status(HttpStatus.CREATED)
            .body(ApiResponse.success(
                message: "Listing created successfully",
                response));
    }
}
```

Fig. 2.20 Car Listing Controller Class

Service

```
public interface CarListingService {
    CarListingResponse createListing(
        CarListingRequest request);
    CarListingResponse getListingById(Long id);
    CarListingResponse updateListing(
        Long id, CarListingRequest request);
    void deleteListing(Long id);
    void markAsSold(Long id);
    CarListingResponse uploadImages(
        Long id, List<MultipartFile> images)
        throws IOException;
    Page<CarListingResponse> getAllListings(
        int page, int size, String sortBy);
    List<CarListingResponse> getFeaturedListings();
    List<CarListingResponse> getSellerListings(
        Long sellerId);
    void boostListing(Long id);
    void incrementViewCount(Long id);
}
```

Fig. 2.21 Car Listing Service

Entity

```
@Entity
@Table(name = "car_listings")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class CarListing {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "seller_id", nullable = false)
    private Long sellerId;

    @NotBlank
    @Column(nullable = false)
    private String brand;

    @NotBlank
    @Column(nullable = false)
    private String model;
}
```

Fig. 2.22 Car Listing Class

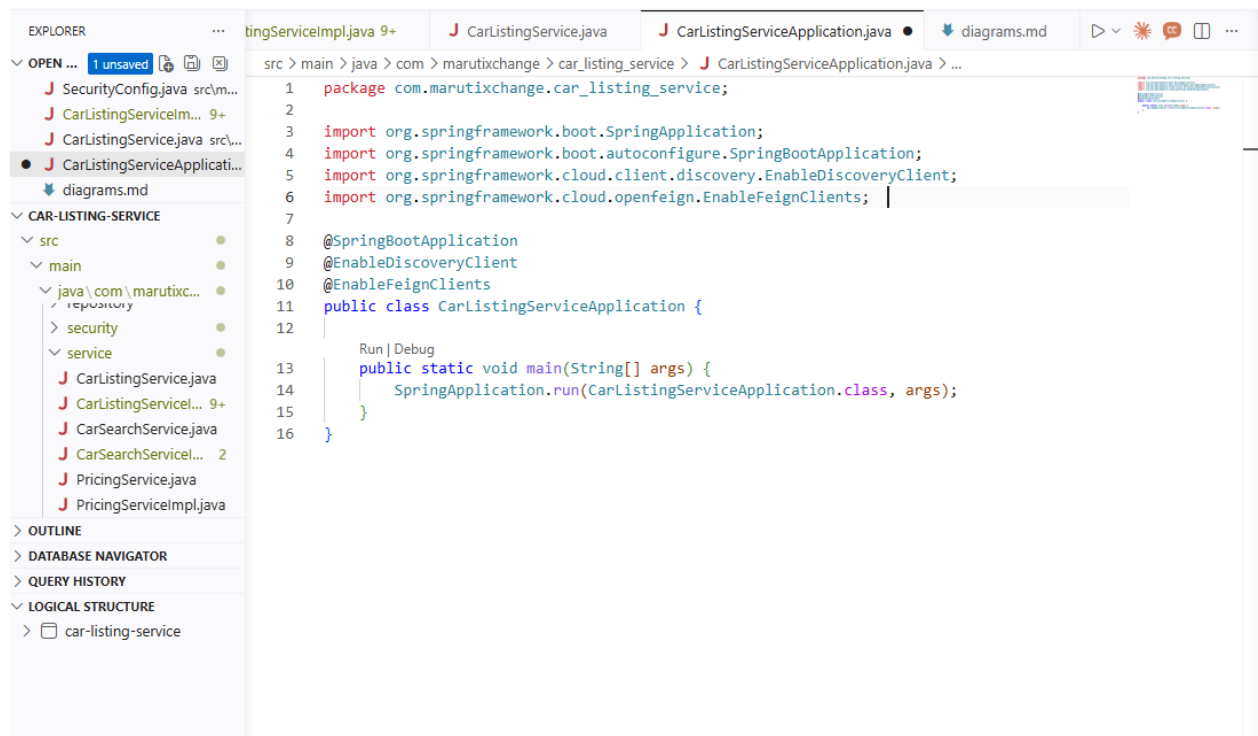


Fig. 2.23 Car Listing Service Application

Rules Engine microservices

```

src > main > java > com > marutixchange > rules_engine_service > controller > J RuleController.java
1  package com.marutixchange.rules_engine_service.controller;
2
3  import com.marutixchange.rules_engine_service.model.RuleContext;
4  import com.marutixchange.rules_engine_service.service.RuleService;
5  import org.springframework.beans.factory.annotation.Autowired;
6  import org.springframework.web.bind.annotation.*;
7
8  @RestController
9  @RequestMapping("/rules")
10 @CrossOrigin(origins = "*")
11 public class RuleController {
12
13     @Autowired
14     private RuleService ruleService;
15
16     @PostMapping("/evaluate")
17     public RuleContext evaluate(@RequestBody RuleContext context) {
18         return ruleService.evaluate(context);
19     }
20 }

```

Fig. 2.24 Controller Class

```

/**
 * Invalid bid amount
 */
rule "BID_INVALID_AMOUNT"
salience 100
when
    $c : RuleContext(type == "BID", ruleMatched == false)
    eval($c.getBidAmount() == null || $c.getBidAmount() <= 0)
then
    $c.setApproved(false);
    $c.setMessage("Invalid bid amount");
    $c.setRuleMatched(true);
end

```

Fig. 2.25 Bidding Rules

```

rule "Reject Old Car"
when
    $c : RuleContext(type == "CAR_LISTING", year < 2000)
then
    $c.setApproved(false);
    $c.setMessage("Car too old");
    $c.setRuleMatched(true);
end

```

Fig. 2.26 Car Rules

```

/**
 * Reject payments below minimum threshold
 */
rule "PAYMENT_MIN_LIMIT_VALIDATION"
when
    $c : RuleContext(type == "PAYMENT", price < 25000)
then
    $c.setApproved(false);
    $c.setMessage("Suspicious amount: cannot be less than 25000");
    $c.setRuleMatched(true);
end

```

Fig. 2.27 Payment Rules


```
// =====
// BLOCK NIGHT NOTIFICATIONS
// =====
rule "BLOCK_NIGHT_NOTIFICATIONS"
salience 100
activation-group "notification-rules"
when
    $c : RuleContext(type == "NOTIFICATION")
    eval($c.getHour() != null && ($c.getHour() > 22 || $c.getHour() < 8))
then
    $c.setApproved(false);
    $c.setMessage("Notifications are disabled between 10 PM and 8 AM");
    $c.setRuleMatched(true);

    System.out.println("[RULE FIRED] BLOCK_NIGHT");
end
```

Fig. 2.28 Notification Rules

2.4.2 Frontend Coding

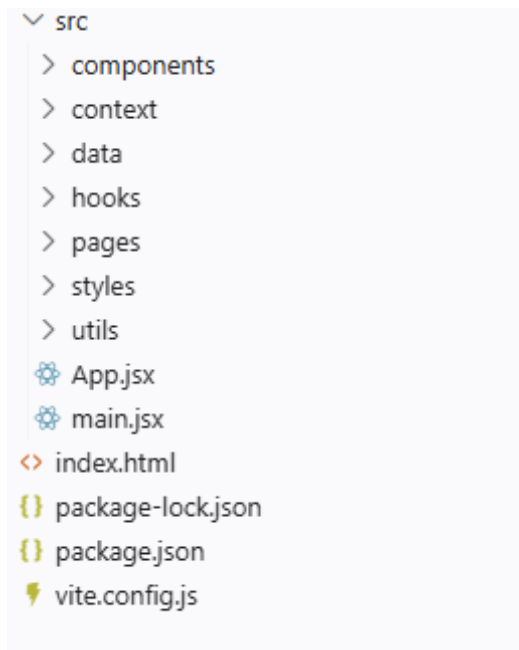


Fig. 2.29 Project Structure

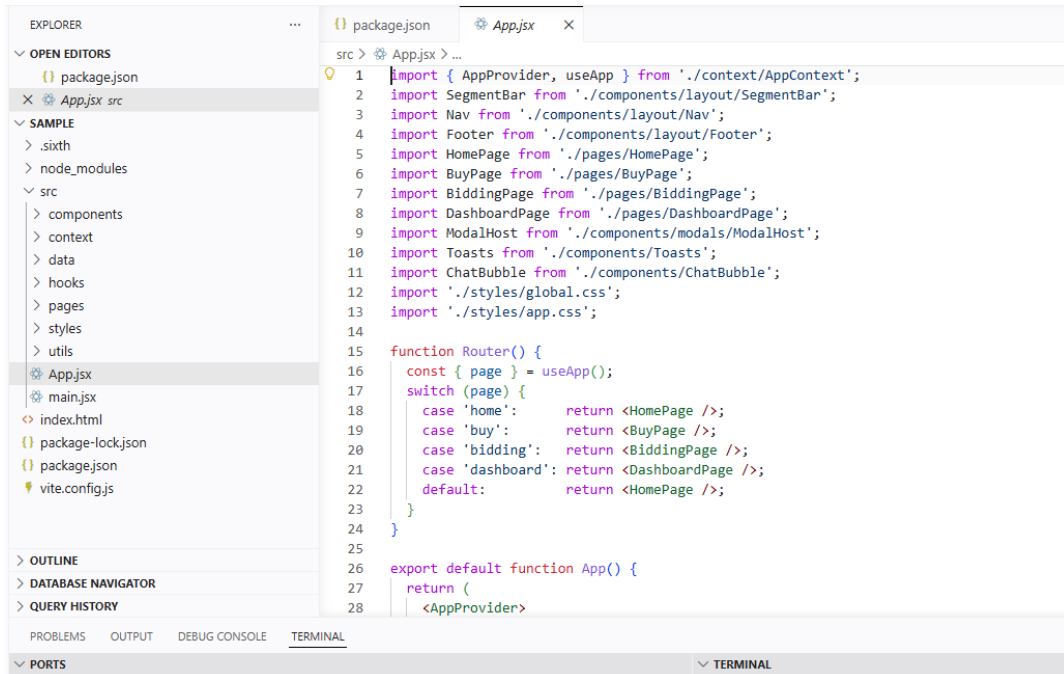


Fig. 2.30 Implementation of App.jsx File in React Application

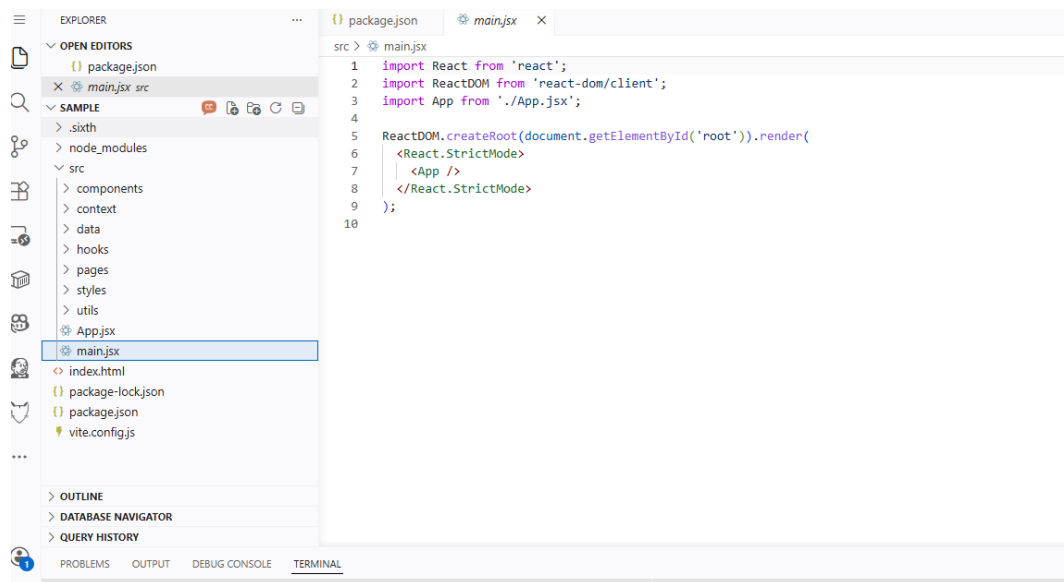


Fig. 2.31 Implementation of main.jsx File as Entry Point of React Application

2.4.3 Database Design and Implementation

This section describes the database design and implementation of the system. The application uses a relational database to store and manage data related to users, cars, bids, and other system functionalities.

1. Database Overview

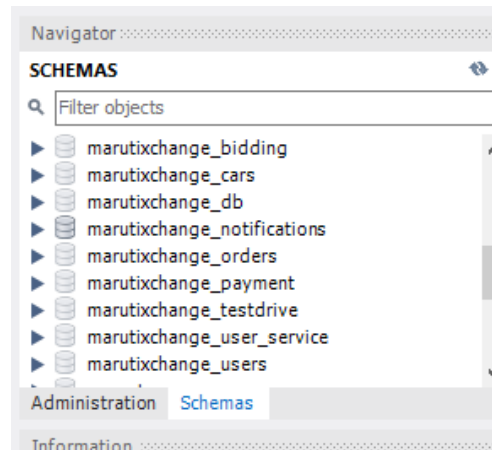


Fig. 2.32 Microservices-Based Database Schemas in MySQL Workbench

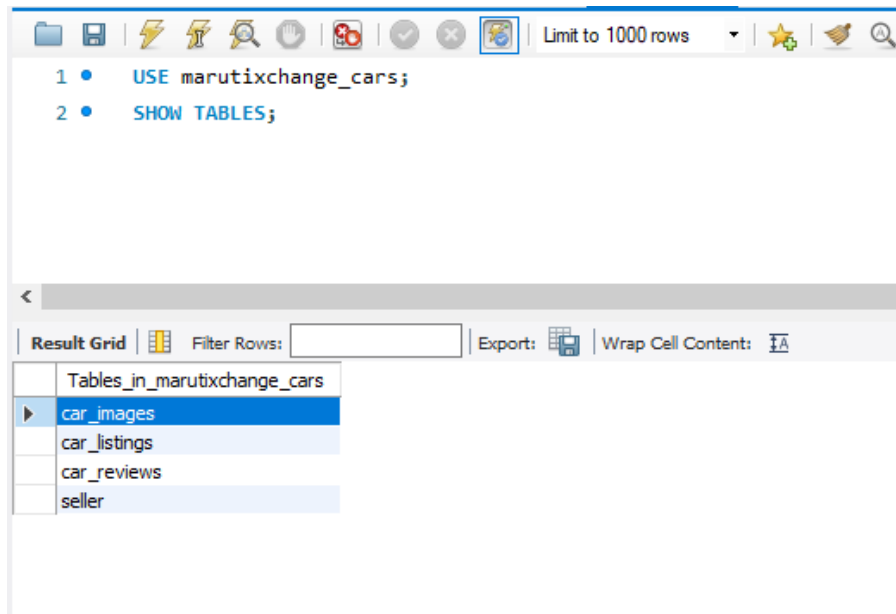


Fig. 2.33 Tables in Car Service Database Schema

Limit to 1000 rows

```

1 • USE marutixchange_cars;
2 • SHOW TABLES;
3 • SELECT * FROM car_listings;

```

	id	asking_price	brand	city	color	created_at	description	engine_cc	fuel_type	inquiry_count	insurance_valid_till
4	650000	Maruti	NULL	NULL	2026-04-01 22:43:25.367742	NULL	NULL	PETROL	0	NULL	0
5	6500000	Maruti	NULL	NULL	2026-04-01 22:44:15.837366	NULL	NULL	PETROL	0	NULL	0
6	650000000	Maruti	NULL	NULL	2026-04-01 22:44:21.781973	NULL	NULL	PETROL	0	NULL	0
7	650000000	Maruti	NULL	NULL	2026-04-01 22:44:23.796692	NULL	NULL	PETROL	0	NULL	0
8	6500000000000000	Maruti	NULL	NULL	2026-04-01 22:44:29.377760	NULL	NULL	PETROL	0	NULL	0
9	650000	Maruti	Delhi	NULL	2026-04-01 23:00:55.551499	NULL	NULL	PETROL	0	NULL	0
10	50000	Maruti	NULL	NULL	2026-04-01 23:13:16.897145	NULL	NULL	PETROL	0	NULL	0
11	50000	Maruti	NULL	NULL	2026-04-01 23:43:40.814131	NULL	NULL	PETROL	0	NULL	0
12	200000	Maruti	NULL	NULL	2026-04-08 19:43:47.895004	NULL	NULL	PETROL	0	NULL	0
13	200000	Maruti	NULL	NULL	2026-04-08 19:52:41.488933	NULL	NULL	PETROL	0	NULL	0

Fig. 2.34 Sample Data from Car Listing

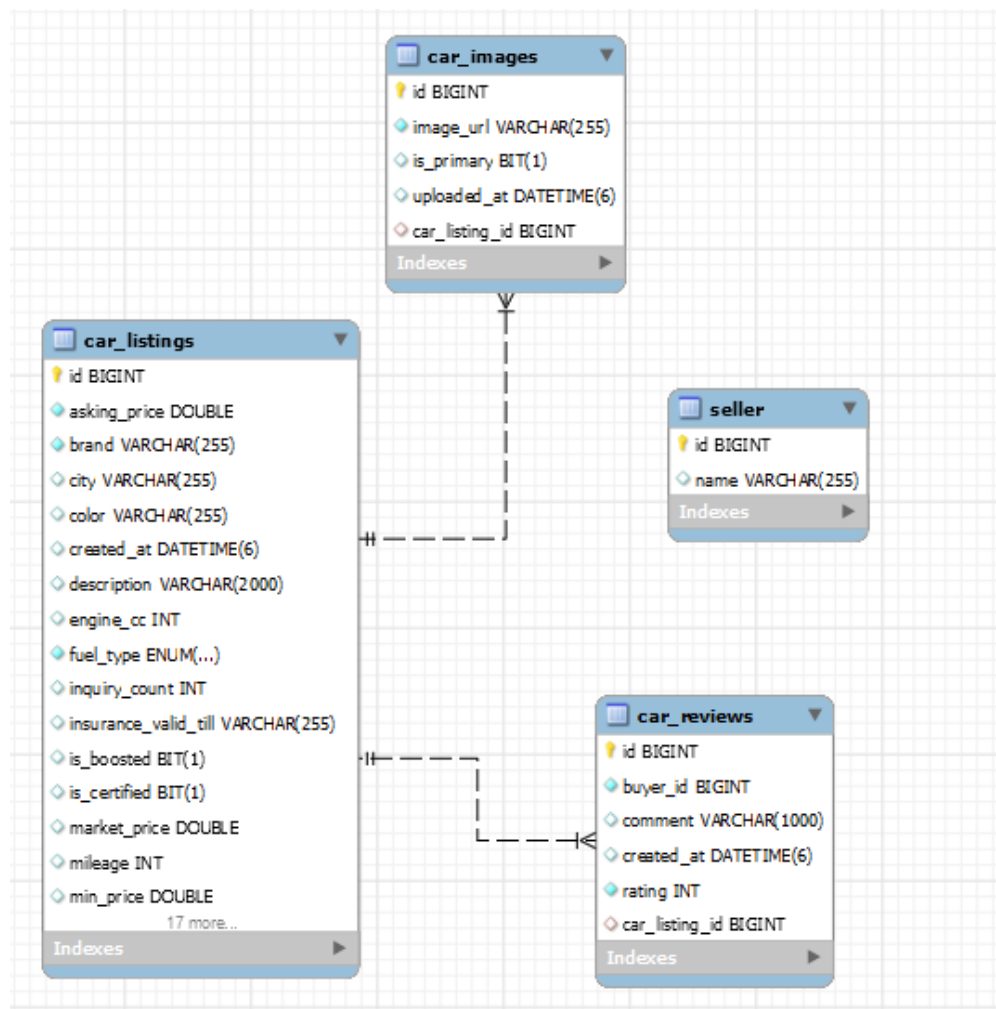


Fig. 2.35 Entity Relationship Diagram of Car Listing Module

Administration Schemas	
Information	
Table: car_listings	
Columns:	
id	bigint AI PK
asking_price	double
brand	varchar(255)
city	varchar(255)
color	varchar(255)
created_at	datetime(6)
description	varchar(2000)
engine_cc	int
fuel_type	enum('PETROL','DIESEL','CNG','ELECTRIC','HYBRID')
inquiry_count	int
insurance_valid_till	varchar(255)
is_boosted	bit(1)
is_certified	bit(1)
market_price	double
mileage	int
min_price	double
model	varchar(255)
owner_number	int
pincode	varchar(255)
rc_number	varchar(255)
registration_number	varchar(255)
seller_id	bigint

Fig. 2.36 Structure of Car Listing Table

Other microservices such as **bidding**, **user service**, **orders**, and **payment** also maintain separate databases to handle their respective functionalities, ensuring modularity and scalability of the system. Each microservice is designed to operate independently, with its own dedicated schema and data management logic. This separation allows different components of the system to be developed, deployed, and maintained without affecting other services.

The bidding service is responsible for managing bid-related operations, including placing bids, storing bid history, and tracking bid status. It ensures that all bid transactions are securely recorded and easily retrievable. The user service manages user-related data such as registration details, authentication information, and user profiles, thereby maintaining secure access control within the system.

Similarly, the orders service handles transaction-related activities, including order placement, tracking, and history management. The payment service manages payment processing, ensuring secure and reliable handling of financial transactions. By separating these functionalities into distinct services, the system achieves better fault isolation, meaning that any failure in one service does not directly impact others.

Furthermore, this microservices-based database design enhances system performance and scalability, as each service can be scaled independently based on demand. It also improves data security and consistency by restricting direct access between services and enforcing well-defined communication through APIs.

Overall, the use of multiple databases across microservices supports flexible, scalable, and maintainable system architecture, making it well-suited for modern full-stack applications.

2.4.4 API Implementation and Testing

This section describes the APIs developed for the system and their testing using Postman. The APIs enable communication between the frontend and backend microservices.

Method: POST

Endpoint: /api/v1/listings

Description: This API is used to add a new car listing to the system. It allows the seller to provide car details such as brand, model, price, and specifications.

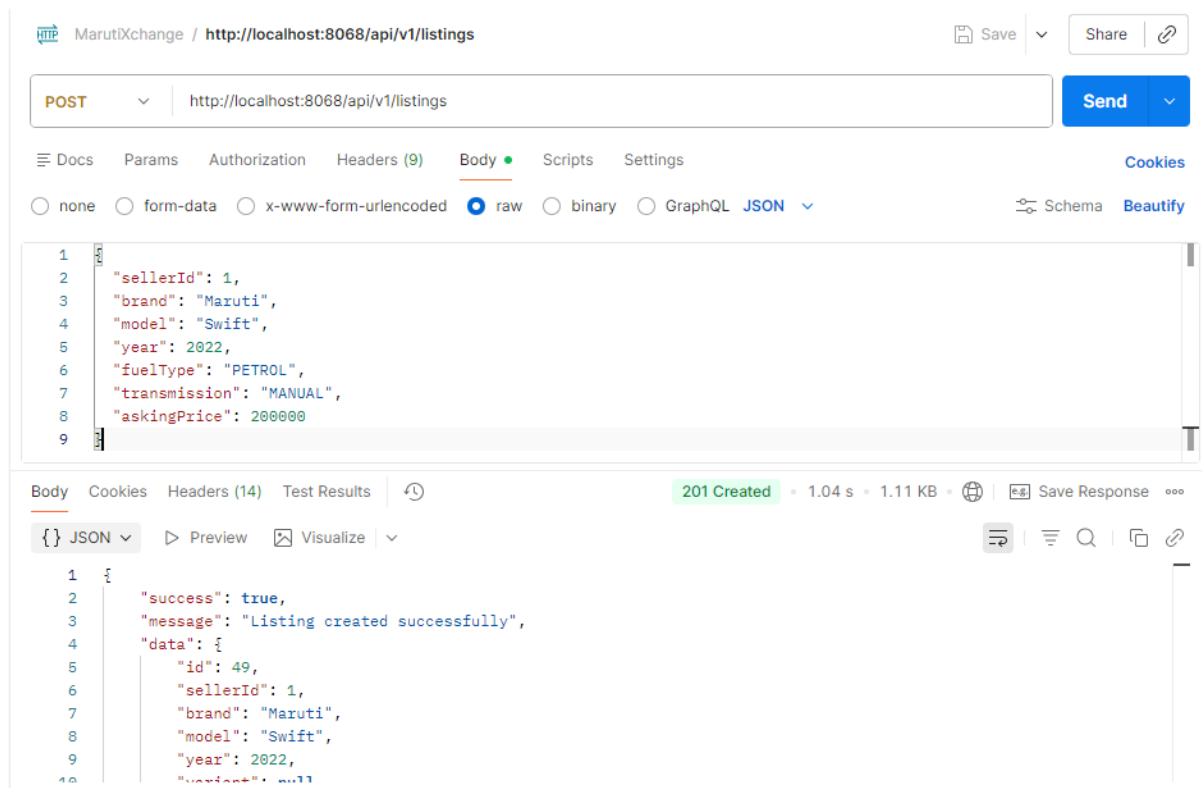


Fig. 2.37 Testing Create Car Listing API using Postman

Method: GET

Endpoint: /api/v1/listings/{id}

Description: This API retrieves the details of a specific car listing based on its unique ID.

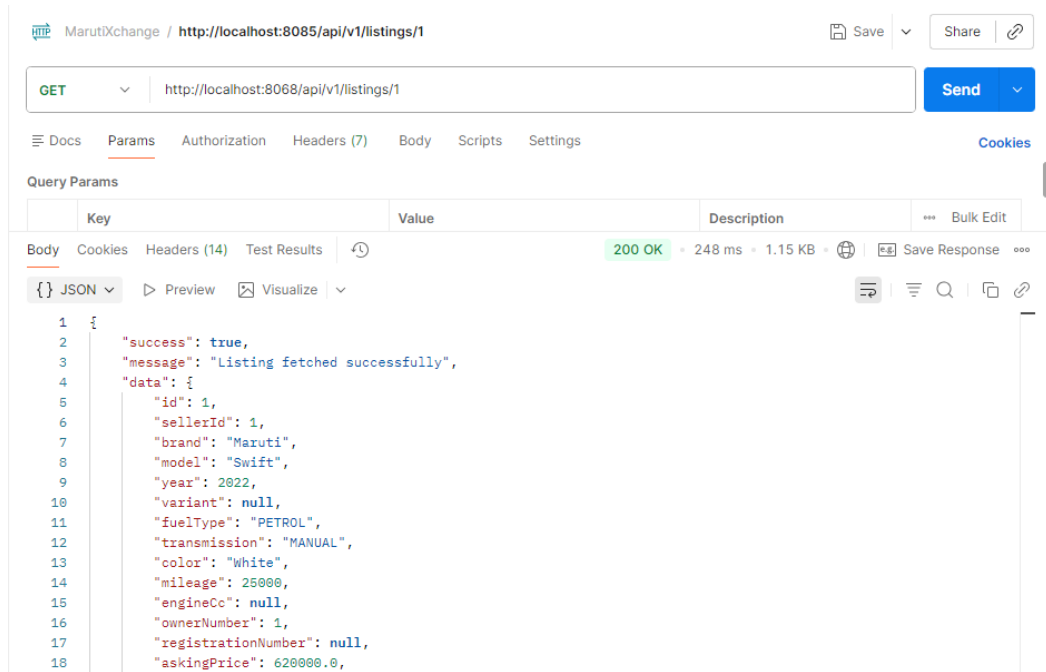


Fig. 2.38 Testing Get Car Listing by ID API using Postman

Method: PUT

Endpoint: /api/v1/listings/{id}

Description: This API is used to update the details of an existing car listing based on its ID.

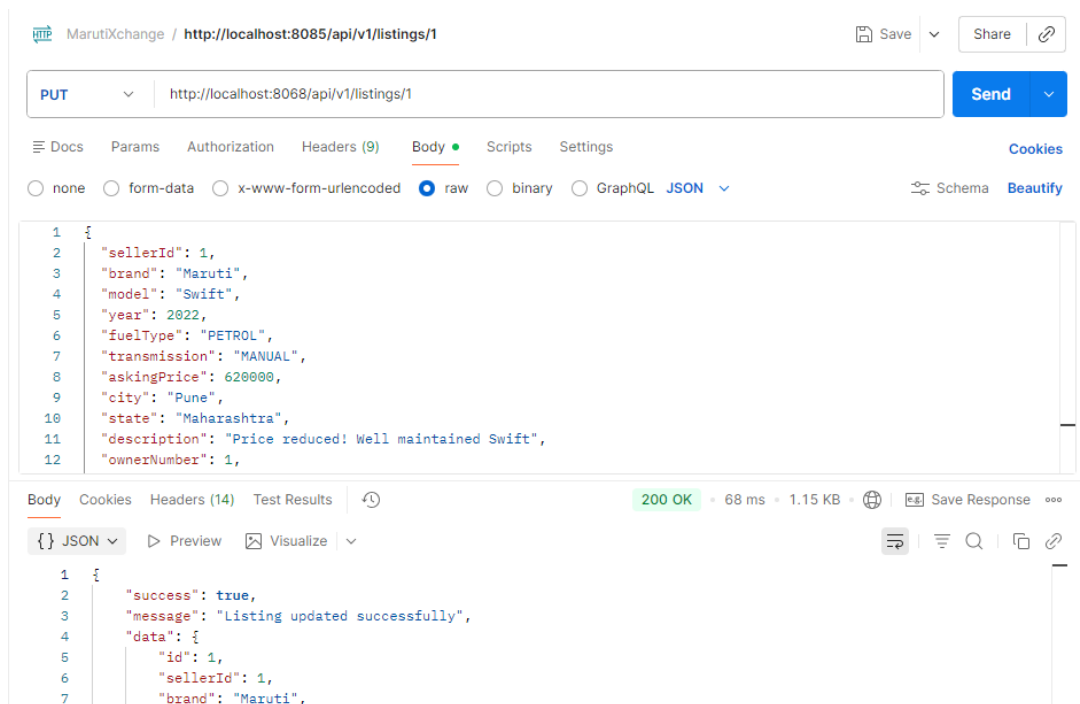


Fig. 2.39 Testing Update Car Listing API using Postman

Method: DELETE

Endpoint: /api/v1/listings/{id}

Description: This API is used to delete an existing car listing from the system using its unique ID.

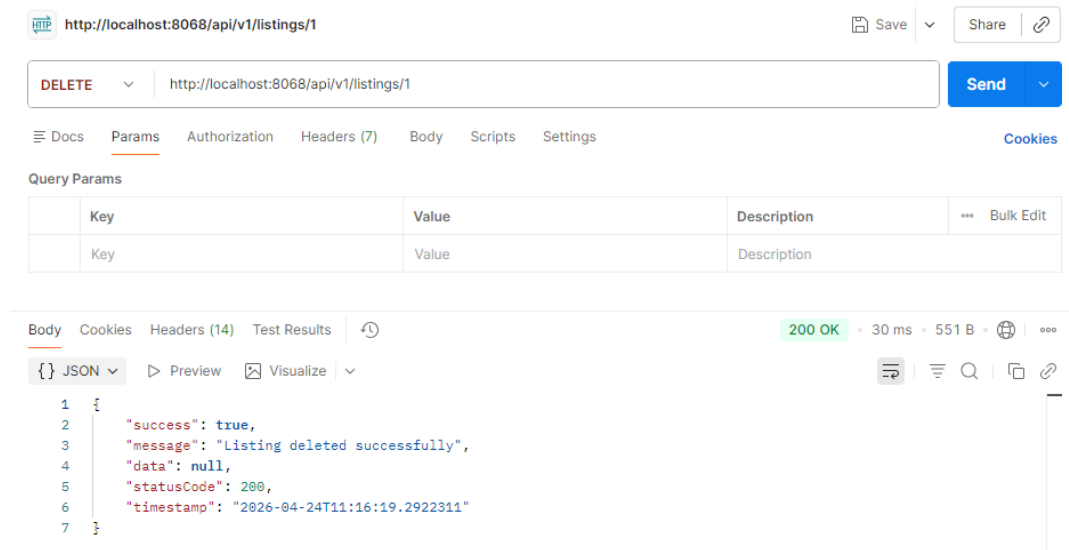


Fig. 2.40 Testing Delete Car Listing API using Postman

In addition to the car listing service, the system includes several other microservices such as bidding, user service, orders, and payment, each responsible for handling specific functionalities. These microservices are designed to operate independently, with their own databases and logic, ensuring better modularity and scalability of the system.

The bidding service manages operations related to placing bids and tracking bid history, while the user service handles user registration, authentication, and profile management. The orders service is responsible for managing transactions and order details, and the payment service ensures secure processing of financial transactions.

Similarly, the orders service manages transaction-related activities such as order placement, tracking, and history management. It ensures that all transactions are processed correctly and stored securely for future reference. The payment service handles financial transactions, including payment processing and verification, ensuring secure and reliable handling of user payments.

All these services communicate through RESTful APIs, enabling smooth data exchange between different components of the system. This architecture allows each service to be developed and deployed independently, improving flexibility and fault isolation.

Overall, the use of multiple microservices enhances system performance, maintainability, and scalability, making the application efficient and reliable for real-world usage.

2.5 SNAPSHOT OF THE PROJECT

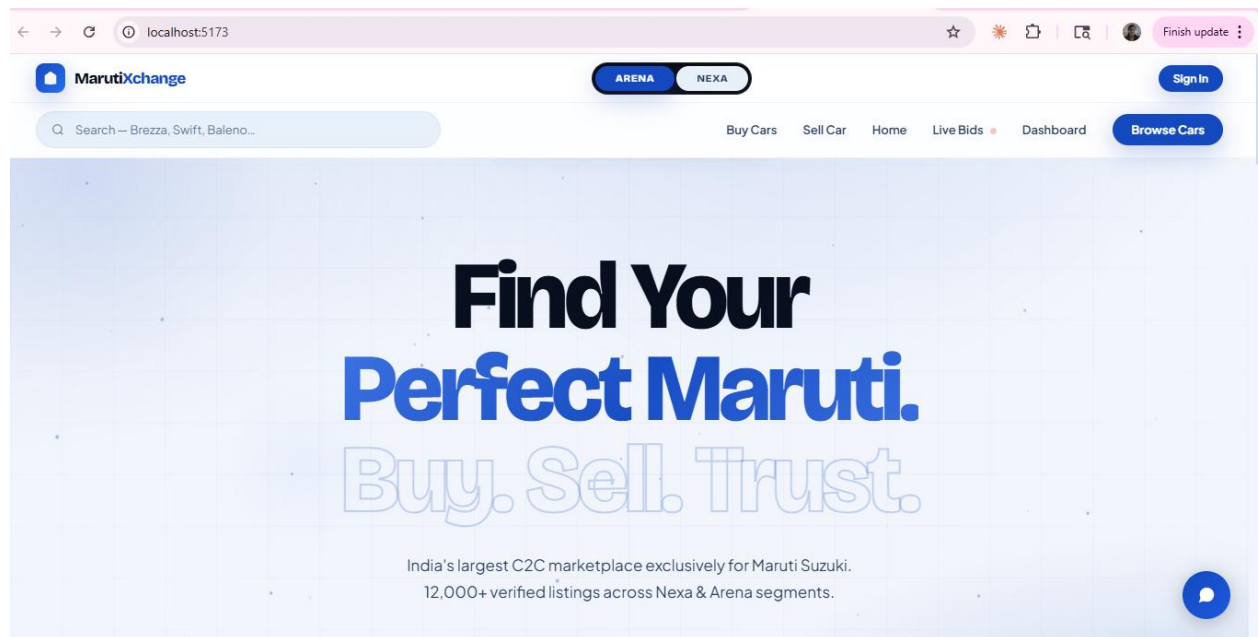


Fig. 2.41 Home Page Interface of MarutiXchange System

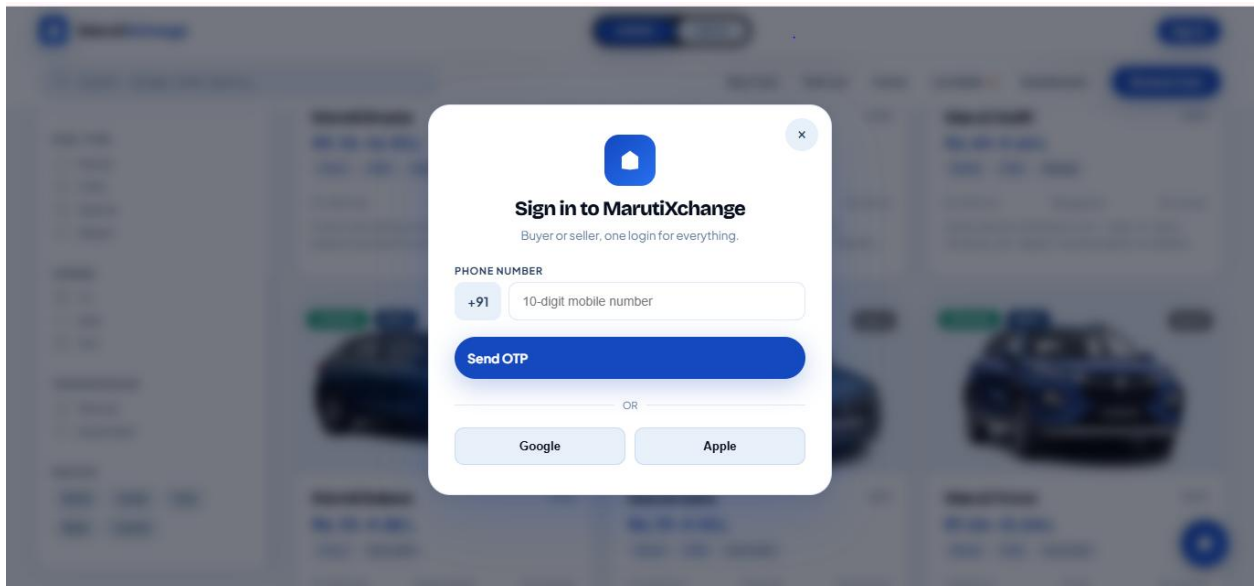


Fig. 2.42 User Login Interface

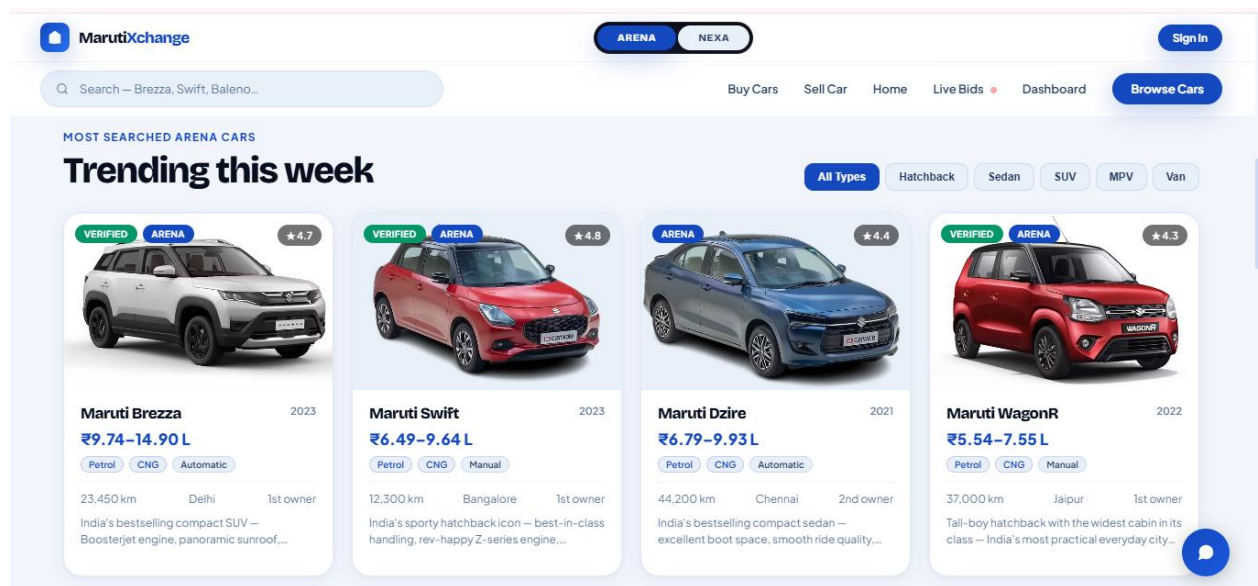


Fig. 2.43 Car Listing Interface Showing Available Vehicles

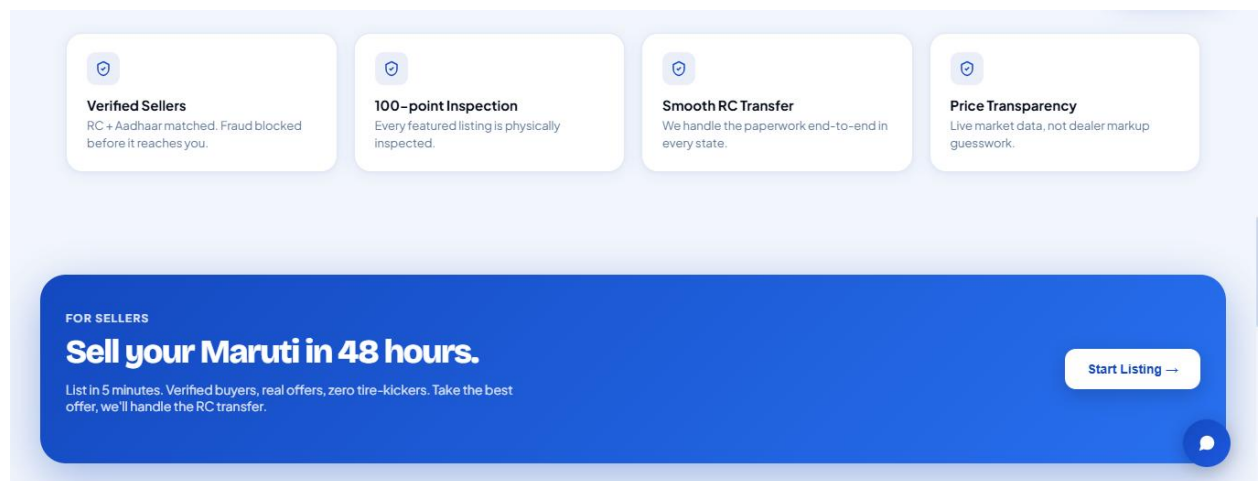


Fig. 2.44 Seller Interface for Listing a Car

List your Maruti

Step 1 of 4 - Car Details

MODEL: Select model

VARIANT: Choose model first

FUEL: Select fuel

YEAR:

KM DRIVEN:

REGISTRATION (E. G. DL 01 AB 1234):

Back Next

Fig. 2.45 Car Listing Form Interface

MarutiXchange

ARENA NEXA

Search — Brezza, Swift, Baleno...

Buy Cars Sell Car Home Live Bids Dashboard Browse Cars

17 cars match: theme: arena

Search by name or city... Sort: Relevance

Filters · 17 results Reset

BODY TYPE

All Types Hatchback

Sedan SUV

MPV Van

PRICE - ₹0 - 40 L

KM DRIVEN - UP TO 100,000

FUEL TYPE

☐ Petrol ☐ CNG

Maruti Brezza 2023

₹9.74-14.90 L

Petrol CNG Automatic

23,450 km Delhi 1st owner

India's bestselling compact SUV — Boosterjet engine, panoramic sunroof, wireless CarPlay &...

Grand Vitara 2022

₹13.70-22.55 L

Petrol Hybrid Automatic

18,200 km Mumbai 1st owner

Premium mid-size SUV with strong hybrid technology and available AllGrip AWD — Maruti's...

Maruti Swift 2023

₹6.49-9.64 L

Petrol CNG Manual

12,300 km Bangalore 1st owner

India's sporty hatchback icon — best-in-class handling, rev-happy Z-series engine, incredible...

Fig. 2.46 Car Search and Filter Interface

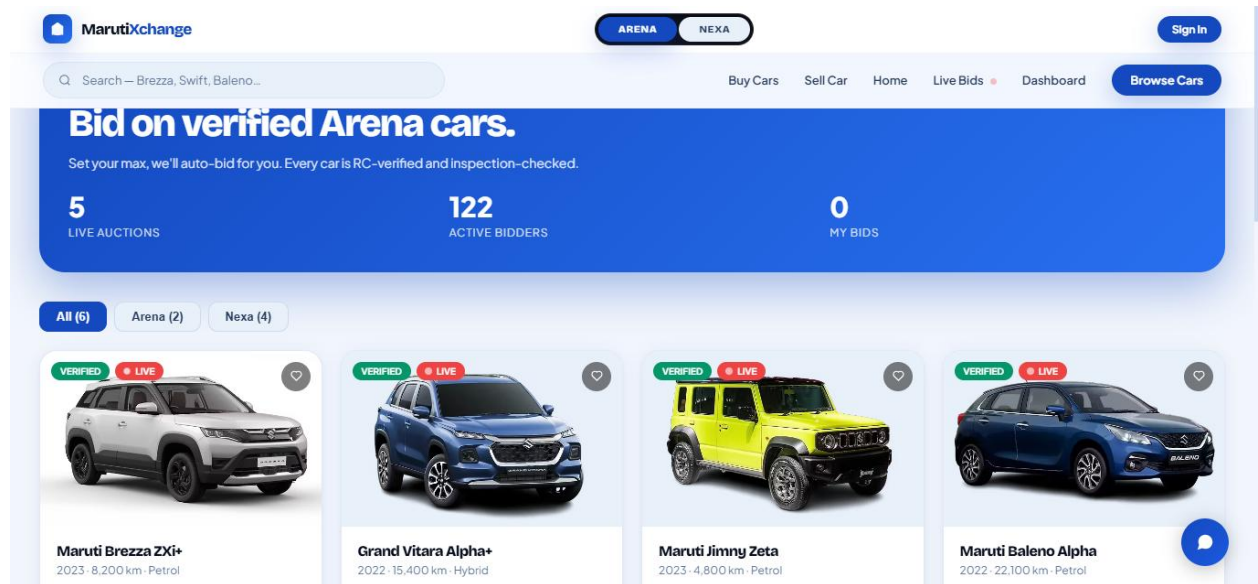


Fig. 2.47 Live Bidding Interface

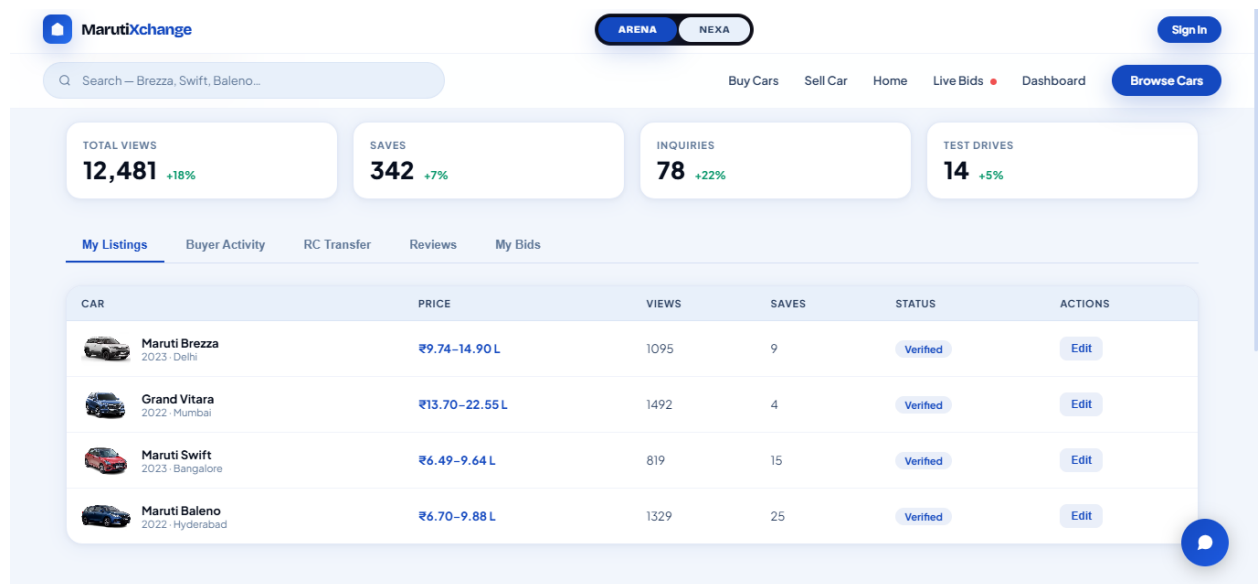


Fig. 2.48 User Dashboard Interface

2.6 APPLICATION OF THE PROJECT

The developed system, *MarutiXchange – Microservices-Based C2C Car Resale Platform*, has multiple real-world applications in the automobile resale industry. It is designed to simplify the process of buying and selling used cars while ensuring transparency, security, and efficiency.

1. Online Car Buying Platform

The system allows users to browse, search, and filter available cars based on various criteria such as price, fuel type, body type, and location. Buyers can easily compare different vehicles and make informed decisions. This eliminates the need for physical visits to multiple dealers and provides a convenient digital marketplace.

2. Car Selling Platform

Users can list their vehicles for sale through a structured and user-friendly interface. The system enables sellers to provide detailed information about their cars, including model, year, mileage, and condition. This helps in attracting genuine buyers and ensures a smooth selling process.

3. Online Bidding System

The platform includes a live bidding feature where users can participate in auctions for listed cars. Buyers can place bids on vehicles, and the system ensures fair competition by maintaining transparency in the bidding process. This feature helps sellers get the best possible price for their vehicles.

4. Secure Authentication System

The system uses OTP-based authentication and supports third-party login options to ensure secure access for users. This enhances user trust and protects sensitive data, making the platform reliable for both buyers and sellers.

5. User Dashboard and Analytics

The platform provides a dashboard where users can track their activities such as listings, bids, inquiries, and test drives. It also displays useful analytics like views and saves, helping users monitor performance and make better decisions.

6. Microservices-Based Scalable Architecture

The system is built using microservices architecture, allowing independent development and deployment of services such as user management, car listing, bidding, and payment. This makes the platform highly scalable, maintainable, and suitable for large-scale applications.

7. Business Rule Validation using Rule Engine

The integration of a rule engine ensures that business rules such as bid validation and pricing constraints are enforced before processing any request. This improves system accuracy and maintains consistency in operations.

8. Secure Payment Integration

The platform supports secure online transactions through payment gateways. This allows users to complete purchases safely and efficiently without relying on offline payment methods.

9. Customer Support and Assistance

The system includes a chat support feature that helps users with queries related to car selection, pricing, and platform usage. This improves user engagement and enhances the overall user experience.

10. Real-World Industry Application

The system can be used by automobile companies, dealerships, and online marketplaces to manage used car transactions digitally. It reduces manual effort, increases transparency, and improves operational efficiency.

2.7 REFERENCES

<https://www.marutisuzuki.com/>
<https://www.nexaexperience.com/>
<https://www.arenaofmaruti.com/>
<https://www.cardekho.com/used-cars>
<https://www.cars24.com/>
<https://www.spinny.com/>
<https://droom.in/>
<https://www.carwale.com/used/>
<https://www.zigwheels.com/used-car>
<https://www.mahindrafirstchoice.com/>
<https://www.olx.in/cars/>
<https://chat.openai.com/>
<https://claude.ai/>